

CC23xx/CC27xx In-Circuit Debugging Tools Guideline

Oslo SW Tools

Exported on 02/15/2026

Table of Contents

1	Table of Contents	8
2	Terms and definitions.....	9
3	Device and Part IDs.....	11
4	Debug sub system	12
4.1	Relevant Ice Melter sequences	13
4.1.1	DORMANT to SWD SEQUENCE.....	13
4.1.2	SWD to DORMANT SEQUENCE.....	13
4.2	Ice Melter state survives resets and power modes.....	14
4.3	SWD Connection Overrides Normal I/O Control of SWDCK/SWDIO pins.	14
4.4	Power up requests	14
4.4.1	Debug power up request (CDBGPWRUPREQ).....	15
4.4.2	System power up request (CSYSPWRUPREQ).....	15
4.5	SWD Access Points	15
4.5.1	CFG-AP DEVICESTATUS (Physical address 0x10C).....	16
5	Boot code	18
5.1	Boot status	18
5.2	SACI mode.....	20
5.2.1	Enter SACI mode	20
5.2.2	Stay in SACI mode	21
5.2.3	Exit SACI mode.....	21
5.2.3.1	Exit SACI, Halt at Bootloader/Application	22
5.2.3.2	Exit SACI, Enter Shutdown Mode	22
5.2.3.3	Exit SACI, Run Bootloader/Application	22
5.2.3.4	Reset.....	22
5.3	CRC Specification	23
6	Connect to target	24
6.1	Connect to SW-DP.....	29

6.2	Connect to CPU.....	30
6.3	Terms regarding connecting to the target.	30
7	Flash programming.....	32
7.1	Non-main flash address ranges.....	33
7.2	Suggested flash programming sequence	33
8	Programming of CCFG User record	37
9	Secure Debug Commands Sequence	38
10	Use cases	40
10.1	Download and debug	40
10.1.1	Use Case.....	40
10.1.2	UI	40
10.1.3	Sequence	40
10.2	Download without debugging	40
10.2.1	Use case	40
10.2.2	UI	40
10.2.3	Sequence	40
10.3	Debug without downloading	41
10.3.1	Use case	41
10.3.2	UI	41
10.3.3	Sequence	41
10.4	Attach to running target	41
10.4.1	Use case	41
10.4.2	UI	41
10.4.3	Sequence	42
10.5	Debug through STANDBY	42
10.5.1	Use case	42
10.5.2	UI	42
10.5.3	Sequence	42
10.6	Debug through reset	42

10.6.1	Use case	42
10.6.2	UI	43
10.6.3	Sequence	43
10.7	Debug through SHUTDOWN	43
10.7.1	Use case	43
10.7.2	UI	43
10.7.3	Sequence	44
10.8	Debugging without system power up request	44
10.8.1	Use case	44
10.8.2	UI	44
10.8.3	Sequence	45
10.9	Debugging a device with SWDCK/SWDIO pins driven by application	45
10.9.1	Use case	45
10.9.2	UI	45
10.9.3	Sequence	45
10.10	Communicate with application through SEC-AP mailbox	46
10.10.1	Use case	46
10.10.2	UI	46
10.10.3	Sequence	46
11	Appendix	47
11.1	SACI commands for CC27xx devices	47
11.1.1	Program SCFG Sector - SACI_CMD_FLASH_PROG_SCFG_SECTOR	47
11.1.1.1	Restrictions	47
11.1.1.2	Side-Effects	47
11.1.1.3	Parameter Words	47
11.1.1.4	Response Words	48
11.1.1.5	Check Sequence	48
11.1.2	Verify Flash SCFG Sector - SACI_CMD_FLASH_VERIFY_SCFG_SECTOR	49
11.1.2.1	Restrictions	49

11.1.2.2	Parameter Words	49
11.1.2.3	Response Words	50
11.1.2.4	Check Sequence	50
11.1.3	Program OTP Sector - SACI_CMD_FLASH_PROG_OTP_SECTOR (REMOVED in PG2.0)	51
11.1.3.1	Availability	51
11.1.3.2	Restrictions	51
11.1.3.3	Side-Effects	51
11.1.3.4	Parameter Words	51
11.1.3.5	Response Words	52
11.1.3.6	Check Sequence	53
11.1.4	Read Secure Boot or HSM FW update Status - SACI_CMD_GET_SECBOOT_HSMFWUPDATE_STATUS.....	53
11.1.4.1	Availability	53
11.1.4.2	Restrictions	53
11.1.4.3	Parameter Words	53
11.1.4.4	Response Words	54
11.1.5	Read the HSM system information - SACI_CMD_HSM_GET_SYS_INFO ..	54
11.1.5.1	Availability	54
11.1.5.2	Restrictions	55
11.1.5.3	Parameter Words	55
11.1.5.4	Response Words	55
11.1.6	Request Debug Key ID - SACI_CMD_DEBUG_REQ_KEY_ID.....	58
11.1.6.1	Availability	59
11.1.6.2	Restrictions	59
11.1.6.3	Parameter Words	59
11.1.6.4	Response Words	60
11.1.6.5	Check Sequence	61
11.1.7	Request Debug Challenge - SACI_CMD_DEBUG_REQ_CHALLENGE	61
11.1.7.1	Availability	62

11.1.7.2	Restrictions	62
11.1.7.3	Parameter Words	63
11.1.7.4	Response Words	63
11.1.7.5	Check Sequence	64
11.1.8	Submit Debug Challenge Response - SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP	64
11.1.8.1	Availability	65
11.1.8.2	Restrictions	65
11.1.8.3	Parameter Words	65
11.1.8.4	Response Words	66
11.1.8.5	Check Sequence	67
11.1.9	Submit Debug Challenge Response - SACI_CMD_DEBUG_CLOSE_SESSION	67
11.1.9.1	Availability	68
11.1.9.2	Restrictions	68
11.1.9.3	Parameter Words	68
11.1.9.4	Response Words	68
11.1.9.5	Check Sequence	69
11.1.10	HSM FW Provision- SACI_CMD_HSM_FW_PROVISION.....	69
11.1.10.1	Availability	69
11.1.10.2	Flow Control Mechanism and Response Handling.....	69
11.1.10.3	Other considerations	69
11.1.10.4	Restrictions	70
11.1.10.5	Parameter Words	70
11.1.10.6	Response Words	70
11.1.10.7	Checks	71
11.1.10.8	Memory Clear	72
11.2	SACI commands for User Record Programming	72
11.2.1	Program CCFG User Record - SACI_CMD_FLASH_PROG_CCFG_USER_REC	72

11.2.1.1	Restrictions	72
11.2.1.2	Parameter Words	72
11.2.1.3	Response Words	73
11.2.1.4	Check Sequence	73
11.2.2	Get CCFG User Record - SACL_CMD_MISC_GET_CCFG_USER_REC	73
11.2.2.1	Availability	73
11.2.2.2	Restrictions	74
11.2.2.3	Parameter Words	74
11.2.2.4	Response Words	74
11.2.2.5	Check Sequence	75

1 Table of Contents

2 Terms and definitions

Term	Description
Ice Melter	Circuit that monitors the serial stream on SWDCK/SWDIO looking for certain patterns. Ice Melter will indicate it states using the SWDSEL and JTAGSEL signals. The state of Ice Melter survives resets and power modes.
FCFG	Factory Configuration Special flash section which is programmed in the factory
CCFG	Customer Configuration Special flash section which can (and must) be programmed by the customer.
SCFG	Security Configuration (CC27xx only) Special flash section which can (and must) be programmed by the customer.
OTP	One Time Programmable sector (CC27xx PG 1.x only) Special flash section with HSM assets.
HSM	Hardware Security Module (CC27xx only) Hardware module that together with SW programmed to a specific HSM region in main flash, support the Secure Boot sequence.
SACI	SEC-AP Command Interface. This interface is used to communicate with the boot code.
SACI mode	It is only possible to use SACI while the boot code is in SACI mode. In this mode the boot code is waiting for SACI commands. The boot code will stay in this mode until the SACI inactivity timeout (if not disabled), or until an exit SACI command has been issued.

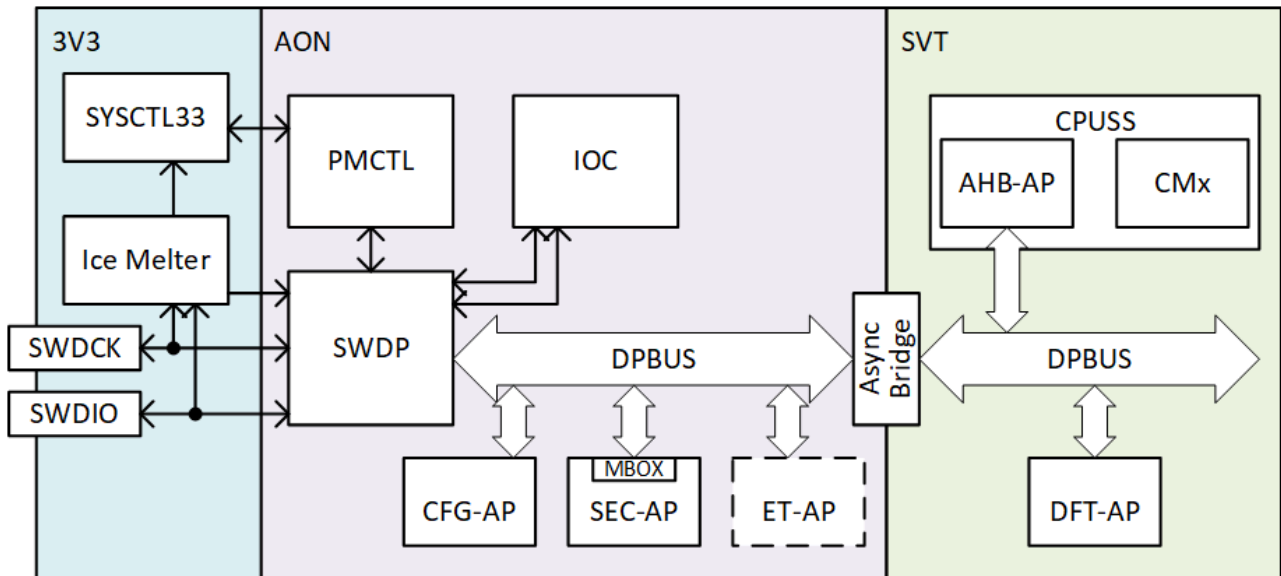
Term	Description
SACI inactivity timeout	To prevent the device to stay in SACI forever (if for example the device enters SACI by accident), a SACI inactivity timeout ensures that the device will exit SACI. The timeout value is specified in FCFG and can be overwritten by CCFG. To disable the timeout, the debugger must send a command to the SEC-AP mailbox before the timeout. Note: The SACI inactivity timeout only needs to be disabled once per SACI session.
Debug probe loop	A loop in the boot code that waits for the debug probe to take control. The boot code enters the debug probe loop after the SACI_CMD_DEBUG_EXIT_SACI_HALT command. See Exit SACI, Halt at Bootloader/Application (see page 22)
Debugger	This generally refers to all the parts (external to the device to be debugged) that is used to debug the device. This includes the debug probe, and the software running on the PC host.
Debug probe	This is the hardware (with associated firmware) that directly interacts with the device to be debugged.

3 Device and Part IDs

Device name	Device Revision	Device ID	Device Part ID
CC2340R5	0x0 PG 1.x 0x1 PG 2.x	0xBB84	Part: 0x2DDA Variant: Don't care
CC2745R10	<0x5 PG 1.x >=0x5 PG 2.x	0xBB98	Part: 0xAF23 Variant: 0xE8
CC2745P10	<0x5 PG 1.x >=0x5 PG 2.x	0xBB98	Part: 0xAF23 Variant: 0xDE
CC2745R7	<0x5 PG 1.x >=0x5 PG 2.x	0xBB98	Part: 0xAF23 Variant: 0x87
CC2745P7	<0x5 PG 1.x >=0x5 PG 2.x	0xBB98	Part: 0xAF23 Variant: 0xAB
CC2755P10	<0x5 PG 1.x >=0x5 PG 2.x	0xBB98	Part: 0x6301 Variant: 0x60

- The Device Revision can be read from CFG-AP:DEVICEID (bit 31:28)
- The Device ID can be read from CFG-AP:DEVICEID (bit 27:12)
- The Device Part ID can be read from CFG-AP:PARTID:PART (bit 15:0) and CFG-AP:PARTID:VARIANT (bit 23:16)

4 Debug sub system



Above shows a functional block diagram for the Debug Sub System in CC23xx/CC27xx devices. See also [Chapter 5 in the Technical Reference Manual](#) [Technical Reference Manual \(TRM\)](#)¹.

Note

Until the CC27xx device has been publicly released to the market, there may not be a TRM to reference. For most but not all TRM reference needs you can reference the CC23xx TRM instead of the CC27xx TRM. When the CC27xx TRM is made available, please use that instead.

A circuit called **Ice Melter** monitors the serial stream on the SWDCK/SWDIO pins looking for certain patterns. Standardized patterns defined by ARM (see [ARM Debug Interface ADIV5.1 Supplement](#)² section 6.3) are used for switching the probe connection mode between these three modes:

- DORMANT
- SWD
- JTAG

Ice Melter recognizes these sequences and at all times provides a SWDSEL and JTAGSEL signal to the SWDP (Serial wire debug port) and more.

¹ <https://www.ti.com/lit/pdf/swcu193>

² <https://documentation-service.arm.com/static/5ed643eaca06a95ce53f92aa?token=>

4.1 Relevant Ice Melter sequences

For CC23xx/CC27xx devices, the SWD interface must be used for in-circuit debugging. To move the connection state in Ice Melter from DORMANT to SWD and from SWD to DORMANT, below sequences must be used.



Note

All sequences are sent with least significant bit (LSB) first.

4.1.1 DORMANT to SWD SEQUENCE

1. Send 0xFF (8 bits)
2. Send 0x19BC0EA2 E3DDAFE9 86852D95 6209F392 (128 bits) - *Selection Alert sequence*
3. Send 0x0 (4 bits)
4. Send 0x1A (8 bits) - *activation code*
5. Send 0x00 (8 bits)
6. Send 1 for 50 clock cycles - *line reset*



The 8 zeroes (0x00) in step 5 (after the *activation code*), are NOT part of the ARM specification.

4.1.2 SWD to DORMANT SEQUENCE

Before sending this pattern, make sure Ice Melter is already in SWD state (SWDSEL==1)

1. Send 1 for 50 clock cycles - *line reset*
2. Send 0xE3BC (16 bits) - *SWD-to-DS select sequence*
3. Send 0xFF (8 bits)



The 8 ones (0xFF) in step 3 (after the *SWD-to-DS select sequence*), are NOT part of the ARM specification.

It does not need to be ones, there just need to be 8 SWD clock cycles. (Actually only 1-2)



CC23xx/CC27xx devices does not support the sequence to change from JTAG mode to DORMANT mode

4.2 Ice Melter state survives resets and power modes

- The detected connection state in Ice Melter is not affected by any kind of resets so the only way to revert to the unconnected state is to power cycle the device or to send a disconnect sequence on SWD.
- In standby power state (both real and fake) AON is powered and the SWDP thus directly accessible. In real standby it is only clocked at 32kHz. In fake standby it is clocked at 24MHz.
- In shutdown power state AON is not powered, but Ice Melter is still active and SYSCTL33 will wake the device up if SWDSEL==1 and there is a clock edge on SWDCK.
- When the device is held in reset Ice Melter is similarly active and will recognize connection request sequences.



Even though the Ice Melter states survives reset, the rest of the debug sub system does NOT. This means that the debugger must reconnect to the SW-DP after a reset (or SHUTDOWN wakeup). See [Connect to SW-DP](#) (see page 29)

4.3 SWD Connection Overrides Normal I/O Control of SWDCK/ SWDIO pins

The two pins, SWDCK and SWDIO, may be used for arbitrary purposes by the running application. Once Ice Melter signals a connected state, PMCTL will take over control of the two pins, SWDCK and SWDIO, bypassing their normal I/O data path through IOC. However, if the application has configured at least one of the SWDCK/SWDIO pins as output, it may not be possible for a probe to apply the required connection sequences. In such cases, the CC23xx/CC27xx device has to be held in reset (during which these two pins are hardwired to be inputs and enabled) and send the connection sequence then.

4.4 Power up requests

Standardized SWD commands exists to override the power state of the device

4.4.1 Debug power up request (CDBGPWRUPREQ)

- CDBGPWRUPREQ will power up and clock the debug domain. The clock will either be 32kHz or a fast clock if it is already running [DEBUGSS-026](#)³
- The debugger must issue a CDBGPWRUPREQ to access any Access Port (AP)

4.4.2 System power up request (CSYSPWRUPREQ)

- CSYSPWRUPREQ will provide the debug domain with a fast clock.
- CSYSPWRUPREQ prevents the device from entering STANDBY power state. Instead a functionally equivalent FAKE_STANDBY power state is entered that keeps SVT powered and clocked.
- If the device is already in STANDBY power state, CSYSPWRUPREQ will wake the device from STANDBY and change the power state to FAKE_STANDBY.
- SHUTDOWN power state does not respect CSYSPWRUPREQ and actually powers off both AON and SVT. However, any edge on the SWDCK pin will wake the device up from SHUTDOWN.
- The debugger must issue a CSYSPWRUPREQ to access AHB-AP

4.5 SWD Access Points

The SWDP converts SWD commands to 32b read/write register accesses over a dedicated debug port bus (DPBUS). The entities that connect to the DPBUS are called *access points* (APs) and each appears as register banks where individual 32b registers can be read/written through SWD operations. Access to each AP can be individually controlled through a firewall mechanism and is under the control of the boot code. The access points supported on CC23xx/CC27xx devices are:

Access Point	Availability	Description
CFG-AP	Always	The read-only CFG-AP provides information about device type in a TI-standardized way to allow a probe to determine what kind of device it is connected to. Additionally unique device IDs may be obtained, (subject to IP area constraints) and current lifecycle and boot software state can be read out.
SEC-AP	Always during boot, optionally during application	SEC-AP implements a bidirectional 32b mailbox that allows the probe to communicate with boot code in a controlled and secure fashion while other APs are locked.

³ <https://confluence.itg.ti.com/requirements/LPRF/DEBUGSS-026>

Access Point	Availability	Description
ET-AP	Configurable	ET-AP, or EnergyTrace++ access point, allows an external probe to capture and timestamp a snapshot of various device state at certain points in time and read them out. This allows state pertinent to the device's power consumption to be logged over time and presented graphically and analyzed on the host side. ET-AP will function independently of whether other APs are locked or not.
AHB-AP	Never during boot, open only for application after correct authentication in PRODDEV/ ENGRDEV lifecycle states	The AHB-AP allows the probe to act as a bus master on the main interconnect and allows it to access anything that is memory-mapped on the device. It also allows control of CPU breakpoints/ watchpoints and can halt the CPU. For security reasons AHB-AP can not be opened until after all hardware locking mechanisms have been applied as CC23xx/ CC27xx devices does not distinguish between CPU and AHB-AP memory accesses. This means that all communication between probe and device during boot to occur using the SEC-AP mailbox. AHB-AP only gets opened once the application starts. Normally a bootloader will also keep AHB-AP closed for the same reason, but it can be opened to allow debugging of the bootloader itself.
DFT-AP	Only in production test lifecycle states	The DFT-AP provides access to test functionality such as PBIST, logic scan, etc. It is always locked for production devices. The DFT-AP actually implements a 4-wire JTAG-TAP instead of connecting to the DPBUS like the other access points.

4.5.1 CFG-AP DEVICESTATUS (Physical address 0x10C)

Bits	Field Name	Description
28	GPIOWAKE	This bit field specify GPIO Wake from shutdown and is driven from PMCTL.
27	SHDWNWAKE	This bit field specify Shutdown wake up status and it is driven from PMCTL.
26	ETAPEN	This bit field specify ETAP Firewall Enable.
25	SECAPEN	This bit field specify SEC-AP Firewall Enable.

Bits	Field Name	Description
24	AHBAPEN	This bit field specify AHBAP Firewall Enable.
23	RXDSTA	Indicates whether the device has written a word to the RXD register, which can be read by the host.
22	TXDSTA	Indicates whether the host has written a word to the TXD register, which can be read by the device.
21	FAKESTANDBY	This bit field gives information about device standby status.
20	STANDBY	This bit field gives information about device standby status.
19	DBGPWRUPACK	This register bit specifies PMCTL acknowledged Debug Powerup Request.
18	SYSPWRUPACK	This bit field specify PMCTL acknowledged system powerup request.
17	JTAGSEL	This bit field specifies about JTAG MODE for TEST TAP is enabled/disabled.
16	SWDSEL	This bit filed specifies whether SWD Connection is enabled/disabled.
15:8	BOOTSTA	Boot state
7:0	LIFECYCLE	This bit field is programmed by boot code with device life cycle information.

5 Boot code

5.1 Boot status

The boot code indicates to the debugger its status through BOOTSTA (8 bits).

BOOTSTA can be accessed in two ways:

- Through CFG-AP: DEVICESTATUS:BOOTSTA (bit 15:8 in the DEVICESTATUS register in CFG-AP)
- Through the CPU: PMCTL:BOOTSTA (bit 7:0 in the BOOTSTA register in PMCTL)

Bit 6 and 7 are sticky. Only a reset can clear them. The boot code uses these two bits to indicate its "mode".

- BOOTSTA[6]=0 and BOOTSTA[7]=0: In the boot code
- BOOTSTA[6]=0 and BOOTSTA[7]=1: In the boot loader
- BOOTSTA[6]=1 and BOOTSTA[7]=1: In the application

Click here to see BOOTSTA values...

A list of boot statuses can be seen in the table below.

The most relevant boot statuses are highlighted with green.

Some boot statuses are terminal, meaning the boot code will stay in this state, these are highlighted in red.

Name	Value	Description
PMCTL_BOOTSTA_BOOT_RESET	0x00	Boot state reset value
PMCTL_BOOTSTA_BOOT_COLD_BOOT	0x01	Starting normal cold boot
PMCTL_BOOTSTA_BOOT_SRAM_REP_DONE	0x02	SRAM repair sequence completed
PMCTL_BOOTSTA_BOOT_GENERAL_TRIMS	0x03	Boot code has started applying general trims
PMCTL_BOOTSTA_BOOT_ENTERED_SACI	0x20	Halt-in-boot into SACI indication
PMCTL_BOOTSTA_BOOT_WAIT_SWD_DISCONNECT	0x36	Waiting for SWD disconnection before device reset
PMCTL_BOOTSTA_BOOT_EXITED_SACI	0x37	Never entered SACI, SACI timed out, or exit from SACI was requested

Name	Value	Description
PMCTL_BOOTSTA_BOOT_WAITLOOP_DBG PROBE	0x38	Waiting for debug-probe (flashless modes)
PMCTL_BOOTSTA_BOOT_FAIL_SRAM_REPAIR	0x3E	SRAM repair failed
PMCTL_BOOTSTA_BOOT_FAULT_HANDLER	0x3F	Fault handler called during boot (before serial bootloader entered)
PMCTL_BOOTSTA_BOOT_COMPLETE	0x80	Boot sequence completed
PMCTL_BOOTSTA_BLDR_WAITLOOP_DBG PROBE	0x81	Waiting for debug-probe to connect
PMCTL_BOOTSTA_BLDR_STARTED	0xBA	Bootloader has started
PMCTL_BOOTSTA_BLDR_CMD_IDLE	0xBB	Bootloader is idle, waiting for a CMD
PMCTL_BOOTSTA_BLDR_CMD_PROCESSING	0xBC	Bootloader has begun processing a CMD
PMCTL_BOOTSTA_BLDR_FAIL_EXECUTION_CONTEXT	0xBD	Bootloader was not started from device boot context
PMCTL_BOOTSTA_BLDR_FAIL_APPTRANSFER	0xBE	Boot ran past transferring control to application (should never happen)
PMCTL_BOOTSTA_BLDR_FAULT_HANDLER	0xBF	Fault handler called during serial bootloader execution
PMCTL_BOOTSTA_BLDR_COMPLETE	0xC0	ROM serial bootloader complete
PMCTL_BOOTSTA_APP_WAITLOOP_DBG PROBE	0xC1	Waiting for debug-probe to connect
PMCTL_BOOTSTA_APP_STARTED	0xC2	Application has started

Name	Value	Description
PMCTL_BOOTSTA_APP_FAIL_NOAPP	0xFD	No application entry-point defined in CCFG (should never happen)
PMCTL_BOOTSTA_APP_FAIL_APPTRANSFER	0xFE	Serial bootloader ran past transferring control to application (should never happen)
PMCTL_BOOTSTA_APP_FAULT_HANDLER	0xFF	Fault handler called after ROM serial bootloader completed

5.2 SACL mode

During boot, the boot code will enter SACL (SEC-AP Command Interface) mode if certain conditions are met (see below). In SACL mode it is possible to communicate with the boot code via the [SEC-AP mailbox](#)⁴, to do things such as providing debug authentication and programming the flash.

Please refer to [SACL \(Sec-AP Command Interface\)](#)⁵ for further description of SACL and the commands it supports.

If `BOOTSTA==PMCTL_BOOTSTA_BOOT_ENTERED_SACL`, it means that the device is in SACL mode.



While in SACL mode, it is not possible to access AHB-AP/CPU.

In general, it is not possible to access AHB-AP/CPU, during the execution of the boot code. The boot code will allow access to AHB-AP depending of the contents of FCFG/CCFG and/or the commands sent to SACL.

5.2.1 Enter SACL mode

Shortly after boot, the boot code will sample SWDSEL in Ice Melter, and if it is set (`SWDSEL==1`), the boot code will enter SACL mode.

The boot code will also enter SACL mode under other conditions, but if `SWDSEL==1` at boot, the device will always enter SACL mode.

To enter SACL the debugger must do two things:

1. Reset the device
2. Make sure `SWDSEL==1` at boot.

⁴ <https://confluence.itg.ti.com/x/vQ75GQ>

⁵ <https://confluence.itg.ti.com/x/g834G>

One way to do this is by doing the following:

1. Reset the device while having a open SWD connection (Reset while SWDSEL==1)
 - a. The suggested reset method is done by writing 0x5C to the RSTCTL register in SEC-AP. This will reset the device while SWDSEL==1, and it does not require any additional line to do a pin reset.
 - i. This will be referred to as "Enter SACL mode using SEC-AP reset"

Another way to enter SACL is to Connect to the SW-DP using pin reset.



Although the device will enter SACL mode, it will not be possible to communicate with SACL (or the rest of the debug sub system), before the debug probe reconnects to the SW-DP (see [Connect to SW-DP \(see page 29\)](#))



It is not possible to connect to the CPU of a running target that requires debug authentication, since the debugger can only provide debug authentication in SACL mode, and the device must be reset to enter SACL mode.

5.2.2 Stay in SACL mode

To stay in SACL the debugger must send any command to the SEC-AP mailbox, after entering SACL mode. This is to disable the *SACL inactivity timeout*.

This command can for example be SACL_CMD_MISC_NO_OPERATION. As the name indicates, this does not do anything, but it does disable the SACL inactivity timeout.

5.2.3 Exit SACL mode

If no commands has been sent to stay in SACL mode, the boot code will exit SACL mode when the SACL inactivity timeout expires.



It is not desirable to use the SACL inactivity timeout as a method to exit SACL mode. The timeout is only intended to exit SACL mode if the device unintentionally enters SACL mode.

Depending on the intent of the debugger and the (previous) state of the device, the debugger can choose between the below methods to exit SACL mode.

5.2.3.1 Exit SACI, Halt at Bootloader/Application

In most scenarios, the suggested method to exit SACI mode, is to use the following command: `SACI_CMD_DEBUG_EXIT_SACI_HALT`

This exit SACI command can be used to configure breakpoints etc. before running the application. This is done by the boot code entering a *Debug probe loop*, where the debugger can do what it wants to do (with access to AHB-AP) before running the application.

The debug probe loop is looping while `R3!=0`, and the boot code will jump to the entry of the bootloader/application after `R3` has been cleared.

The debugger must therefore do the following:

- Wait until `BOOTSTA` equals one of these values:
 - `0x81` (`BLDR_WAITLOOP_DBGPROBE`) = The boot code is waiting before bootloader entry (CCFG restrictions not yet applied)
 - `0xC1` (`APP_WAITLOOP_DBGPROBE`) = The boot code is waiting before application entry
- Halt the CPU
- Optional: Configure breakpoints. For example at start of `main()` or at the start of the application reset vector
- For the current revision of the boot code, the debugger should set the `BOOTSTA` to `0x82/0xC2`, so it does not think the device is in the debug probe loop in the future.
- Write CPU register `R3 = 0x00000000`
- Run the CPU
- Wait until the CPU has reached the breakpoint, if any

5.2.3.2 Exit SACI, Enter Shutdown Mode

If the device wakes up from shutdown due to SWDCK activity, the debugger can issue the following command to return to shutdown: `SACI_CMD_DEBUG_EXIT_SACI_SHUTDOWN`

If `CFGAP.DEVICESTATUS.SHDOWNWAKE=1` and `CFGAP.DEVICESTATUS.GPIOWAKE=0` then the device woke up due to SWD activity.

5.2.3.3 Exit SACI, Run Bootloader/Application

To exit SACI mode and immediately run the application, the debugger can use the following command: `SACI_CMD_BLDR_APP_EXIT_SACI_RUN`

This can for example be used if debugging is not allowed. Flash loading tools can use this method to exit SACI and run application.

5.2.3.4 Reset

Another way to "exit" SACI mode is to issue a reset, but the reset must be issued after the debug probe has disconnected. `SWDSEL==0`, otherwise the device will enter SACI mode again after the reset.

This can be done in two ways:

1. Disconnect from SW-DP (setting SWDSEL == 0) → issue pin-reset
2. Send SACL_CMD_BLDR_APP_RESET_DEVICE command (with option to wait for disconnection before issuing reset) → Disconnect from SW-DP (setting SWDSEL == 0) → The device will be reset (after disconnecting).

5.3 CRC Specification

Whenever the use of a CRC32 is mentioned throughout this document, the following CRC specification is used for the calculation/verification of the CRC32.

- [CRC32/ISO-HDLC](#)⁶
 - CRC32_INIT = 0xFFFFFFFF
 - CRC32_POLY = 0x04C11DB7
 - CRC32_RPOLY = 0xEDB88320
 - CRC32_FINAL = 0xFFFFFFFF

⁶ <https://reveng.sourceforge.io/crc-catalogue/all.htm#crc.cat.crc-32-iso-hdlc>

6 Connect to target

In most scenarios, connecting to the target consists of the following steps and sub steps:

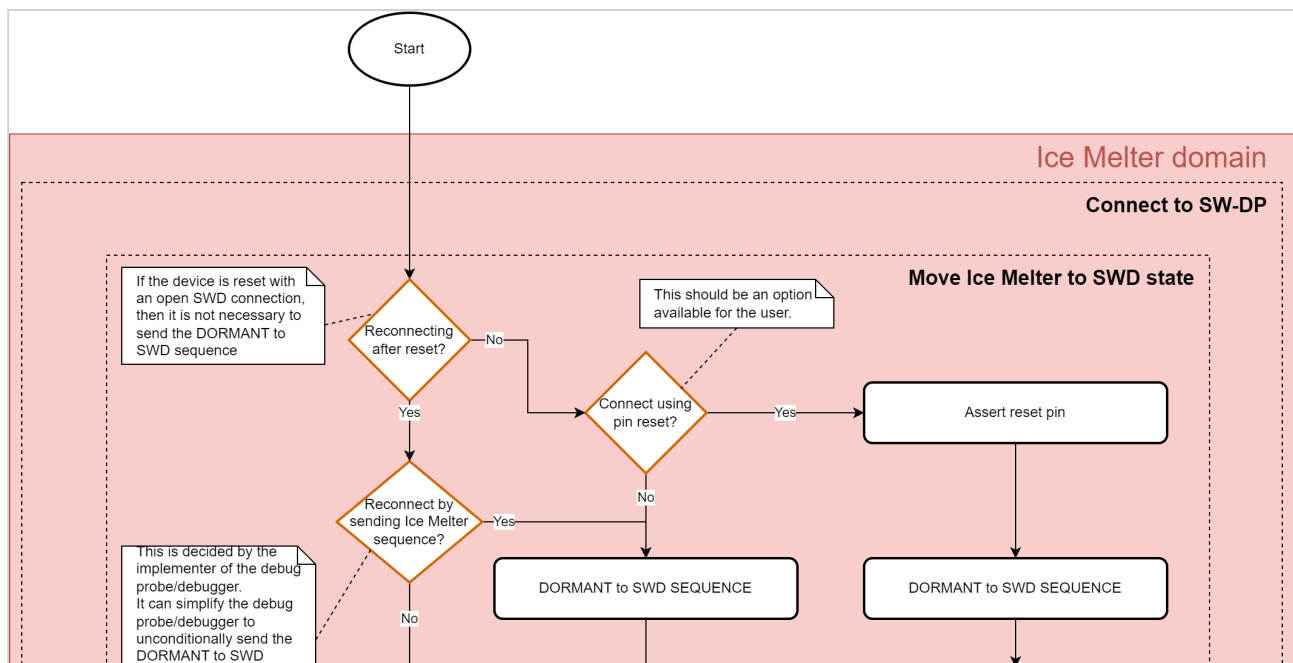
1. Connect to SW-DP
 - a. Including (if not reconnecting after reset): Move Ice Melter to SWD state
2. Connect to CPU, if necessary.
 - a. Submit Authentication, if necessary.
 - b. Exit Debug Probe Loop, if necessary.

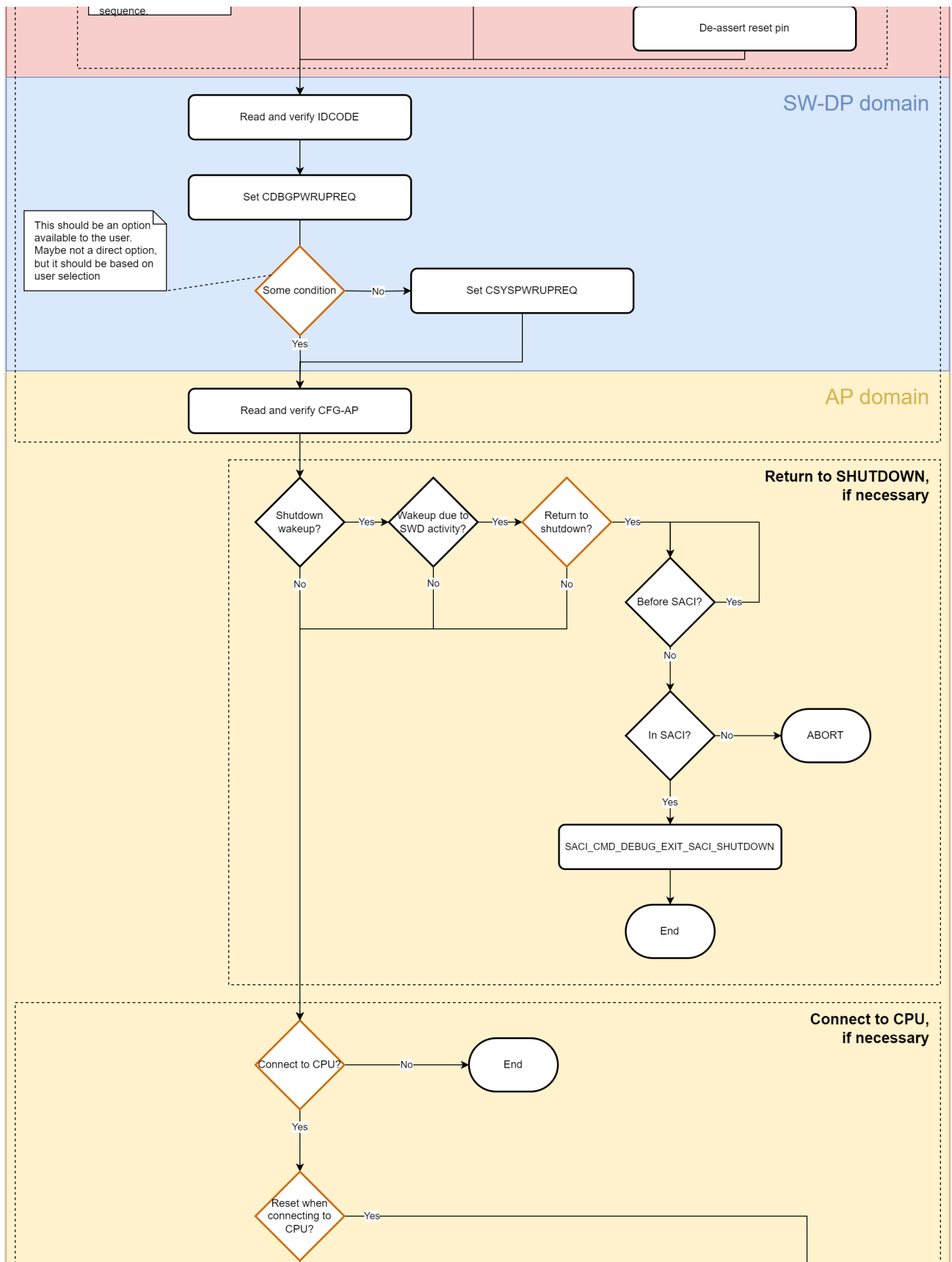
This can be seen on the flow chart below.

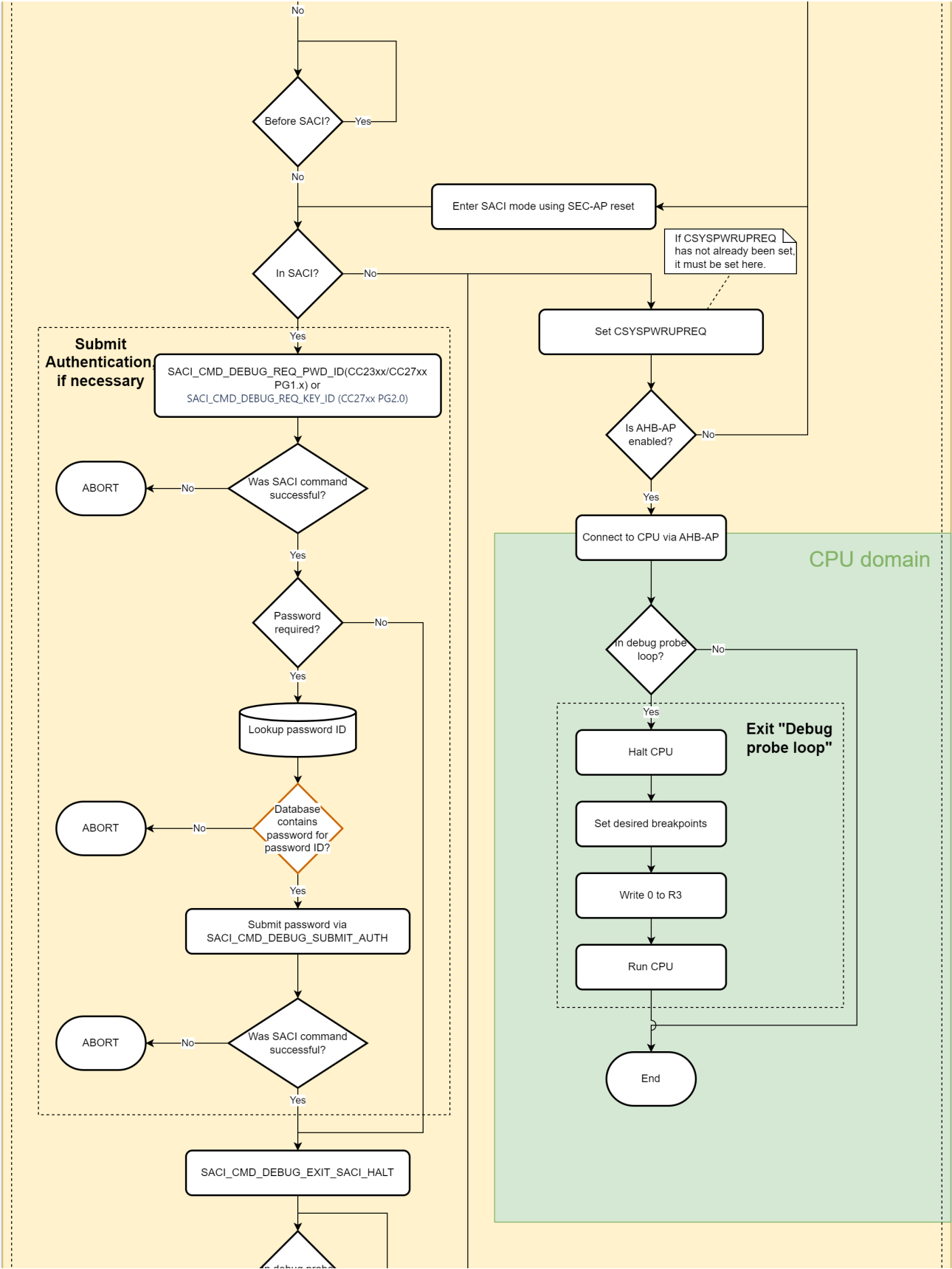
i The flowchart indicates what the debugger needs to check and/or do, NOT what happens on the target.

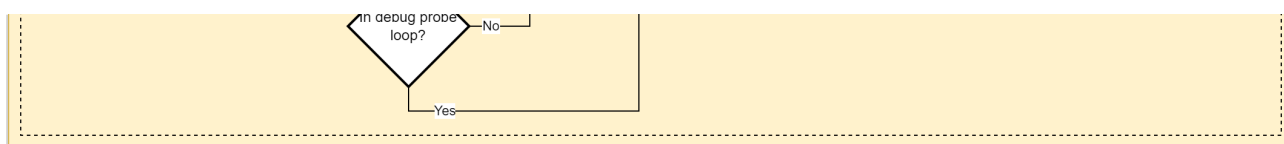
The flow chart is divided into the following "domains":

- Ice Melter domain: Low level sequences using SWDCK, SWDIO and reset pin to move Ice Melter between states.
- SW-DP domain: Direct SW-DP register access only. No APs are being accessed.
- AP domain: Direct AP register access only. Does not require access to AHB-AP/CPU.
- CPU domain: CPU access through AHB-AP. Require access to AHB-AP/CPU. Does not require access to other AP's than AHB-AP.









As shown on the flow chart, the debugger must make some decisions. Some decisions are based on the target state or response, indicated by black borders on the decision blocks. Other decisions are based on the state/mode/implementation/user options of the debugger. These decisions are indicated with an orange border on the decision blocks.

The decisions based on the target state/response (black borders) are described below:

Decision on flow chart	Symbolic equivalent
Shutdown wakeup?	CFGAP:DEVICESTATUS:SHDWNWAKE==1
Wakeup due to SWD activity?	CFGAP:DEVICESTATUS:GPIOWAKE==0
Before SACI?	CFGAP:DEVICESTATUS:BOOTSTA < PMCTL_BOOTSTA_BOOT_ENTERED_SACI
In SACI?	CFGAP:DEVICESTATUS:BOOTSTA == PMCTL_BOOTSTA_BOOT_ENTERED_SACI
Was SACI command successful?	response.result==SUCCESS. (where <i>response</i> is the response of the previous SACI command)
Password required?	response.dataWordCount == 2 (where <i>response</i> is the response of the SACI the previous SACI command: SACI_CMD_DEBUG_REQ_PWD_ID)
In debug probe loop? (in AP domain)	(CFGAP:DEVICESTATUS:BOOTSTA == PMCTL_BOOTSTA_BOOT_WAITLOOP_DBGPROBE) (CFGAP:DEVICESTATUS:BOOTSTA == PMCTL_BOOTSTA_BLDR_WAITLOOP_DBGPROBE) (CFGAP:DEVICESTATUS:BOOTSTA == PMCTL_BOOTSTA_APP_WAITLOOP_DBGPROBE)
In debug probe loop? (in CPU domain)	(PMCTL:BOOTSTA == PMCTL_BOOTSTA_BOOT_WAITLOOP_DBGPROBE) (PMCTL:BOOTSTA == PMCTL_BOOTSTA_BLDR_WAITLOOP_DBGPROBE) (PMCTL:BOOTSTA == PMCTL_BOOTSTA_APP_WAITLOOP_DBGPROBE)

Decision on flow chart	Symbolic equivalent
Is AHB-AP enabled?	CFGAP:DEVICESTATUS:AHBAPEN == 1

The decisions based on the debugger (orange borders) are described below:

Decision on flow chart	Symbolic equivalent/description
Reconnecting after reset?	If the device is reset with an open SWD connection, then Ice Melter will already be in the SWD state, and it is not necessary to send the DORMANT to SWD sequence. So if a reset (or SHUTDOWN wakeup) has happened with an open SWD connection, then the 'yes' branch should be used, otherwise the 'No' branch should be used.
Reconnect by sending Ice Melter sequence?	It is allowed to send the DORMANT to SWD sequence even though Ice Melter is already in SWD state. It can simplify the debug probe/debugger to unconditionally send the DORMANT to SWD sequence, because then there would be no difference between reconnecting and connecting. This is decided by the implementer of the debug probe/debugger.
Connect using pin reset?	To ensure being able to connect to the target in cases where the SWDCK/SWDIO pins are driven by the application, there should be an option to assert the reset pin while sending the DORMANT to SWD sequence.
Some condition	Depending on the purpose of the connection, the system power up request may or may not need to be issued. (In most cases it does need to be issued)

Decision on flow chart	Symbolic equivalent/description
Return to shutdown?	If the target was woken up from SHUTDOWN due to SWD activity, the debugger must decide whether or not to re-enter SHUTDOWN. If for example the debugger's only intent with the SWD activity was to pull the status of the device, then it is probably desirable to re-enter SHUTDOWN mode. The only "normal" shutdown wakeup condition is level-based on I/O. Thus if a real wakeup event occurs between the wakeup due to SWD activity and re-entering SHUTDOWN mode, the device will wake up again immediately, this time indicating an I/O wakeup. If for example the debugger wants to halt the CPU, it must wake up the device to do so, and then it is not desirable to re-enter SHUTDOWN mode.
Connect to CPU?	Depending on the purpose of the connection, it may or may not be desired to connect to the CPU. If for example the debugger wants to utilize the SACI mode, the debugger must only connect to SW-DP, since it is not possible to connect to the CPU.
Reset when connecting to CPU?	If the debugger want to reset the device when connecting to the CPU, it should follow the 'Yes' branch, otherwise it should follow the 'No' branch.
Database contains password for password ID	The debugger must contain some kind of database of passwords. If the debuggers database does not contain the password ID returned by the target for the request password ID SACI command, the 'No' branch should be used. If the database does contain the password ID, the 'Yes' branch should be used.

6.1 Connect to SW-DP

1. If not reconnecting after a reset (or SHUTDOWN wakeup): Send DORMANT to SWD SEQUENCE.
 - a. If user option specifies it, the reset pin should be asserted while sending the sequence.
2. Read IDCODE SW-DP register
3. Verify IDCODE
 - a. The JEDEC Designer ID for Texas Instruments is: 0x17
4. Issue debug power up request

5. Read CFG-AP to determine device
 - a. CFGAP.DEVICEID and CFGAP.PARTID is used to identify the device.

6.2 Connect to CPU

To be able to connect the CPU you first need to be connected to the SW-DP.

You also need to have provided debug authentication, if necessary.

6.3 Terms regarding connecting to the target.

Term	Description
Connect to SW-DP	This refers to following above flowchart and using the 'No' branch of the "Connect to CPU?" decision block.
Reconnect to ... after reset	This refers to following above flowchart and using the 'Yes' branch of the "Reconnecting after reset?" decision block and using the 'No' branch of the "Connect to CPU?" decision block.
Connect to CPU	This refers to following above flowchart and using the 'Yes' branch of the "Connect to CPU?" decision block. If the debugger is already connected to the SW-DP, this refers to following above flowchart, but starting at the 'Yes' branch of the "Connect to CPU?" decision block.
Reconnect to ... after reset	This refers to following above flowchart and using the 'Yes' branch of the "Reconnecting after reset?" decision block.
Connect to ... using pin reset	This refers to following above flowchart and using the 'Yes' branch of the "Connect using pin reset?" decision block.
Connect to SW-DP without system power up request	This refers to following above flowchart and using the 'No' branch of the "Some condition" decision block. The default is to use system power up request, so if not specified, it should be assumed that the 'Yes' branch should be used. If connecting to the CPU after connecting to the SW-DP without system power up request, the system power up request must be issued.

7 Flash programming

On CC23xx/CC27xx devices, flash programming must be done via SACL. No target flash loader should be loaded to RAM.

For more details about flash programming and the SACL commands, see Chapter 8 of the [Technical Reference Manual \(TRM\)](#)⁷

For internal users the more details can be found in the Flash Programming Command section in [SACL \(Sec-AP Command Interface\)](#)⁸.

The most important details are summarized here:

1. The only way to erase flash is to erase the entire MAIN flash + CCFG. For CC27xx devices along with CCFG both SCFG and OTP sectors should be erased.
 - a. With the exception that CCFG can specify MAIN flash sectors to be retained. These sectors will be retained if the *retainSelMainSectors* flag is set in the SACL_CMD_FLASH_ERASE_CHIP command.
 - b. You cannot erase individual sectors.
 - c. This means that CCFG, SCFG and OTP will need to be erased when you want to load a new program to flash, since the only way to erase MAIN flash sectors is to also erase the CCFG, SCFG and OTP sectors.
 - d. Above means that all applications must include a (valid) CCFG sector. For CC27xx devices all applications must include both a valid CCFG and SCFG sectors.
2. (In most cases) You cannot exit SACL using the exit SACL commands, after having programmed an application.
 - a. SACL_CMD_DEBUG_EXIT_SACL_HALT: Cannot be used if CCFG has been modified since the last reset. For CC27xx devices the same apply for SCFG.
 - b. SACL_CMD_BLDR_APP_EXIT_SACL_RUN: Cannot be used if SACL_CMD_FLASH_ERASE_CHIP and/or SACL_CMD_FLASH_PROG_CCFG_SECTOR has been used since the last reset.
 - c. (SACL_CMD_DEBUG_EXIT_SACL_SHUTDOWN: Can only be used if device has been woken up from shutdown due to SWDCK activity)
3. To exit SACL after flash programming, the debugger should use one of below sequences
 - a. Exit SACL mode by first re-entering SACL mode. This is recommended when debugging the application after program load.
 - i. Reset the device
 - ii. The device enters SACL mode, because of the open SWD connection
 - iii. Reconnect to the SW-DP
 - iv. Exit SACL mode.
 - b. Reset device after disconnecting debugger. This is recommended for a standalone flash programming, where it is not intended to debug the application after program load.

⁷ <https://www.ti.com/lit/pdf/swcu193>

⁸ <https://confluence.itg.ti.com/x/g834G>

- i. Issue `SACI_CMD_BLDR_APP_RESET_DEVICE`, with `waitForSwdDisconnect=1`
- ii. Send SWD disconnect sequence
 1. After the disconnect, the boot code will reset the device, and the boot code will not enter SACI mode because `SWDSEL==0`

7.1 Non-main flash address ranges

Programming of non-main flash sectors are supported with specific SACI commands as shown in [Suggested flash programing sequence](#) (see page 33).

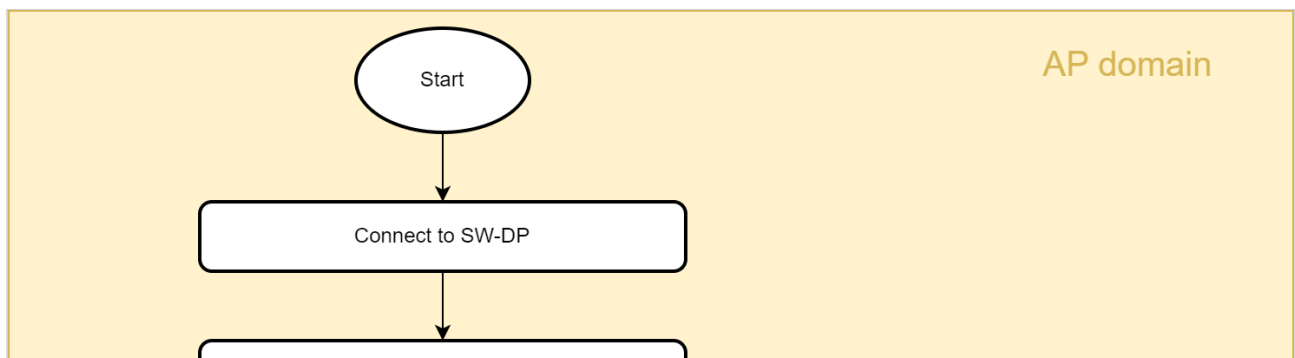
The address range of the different sectors must be used to determine the SACI commands to use for programming of the sectors.

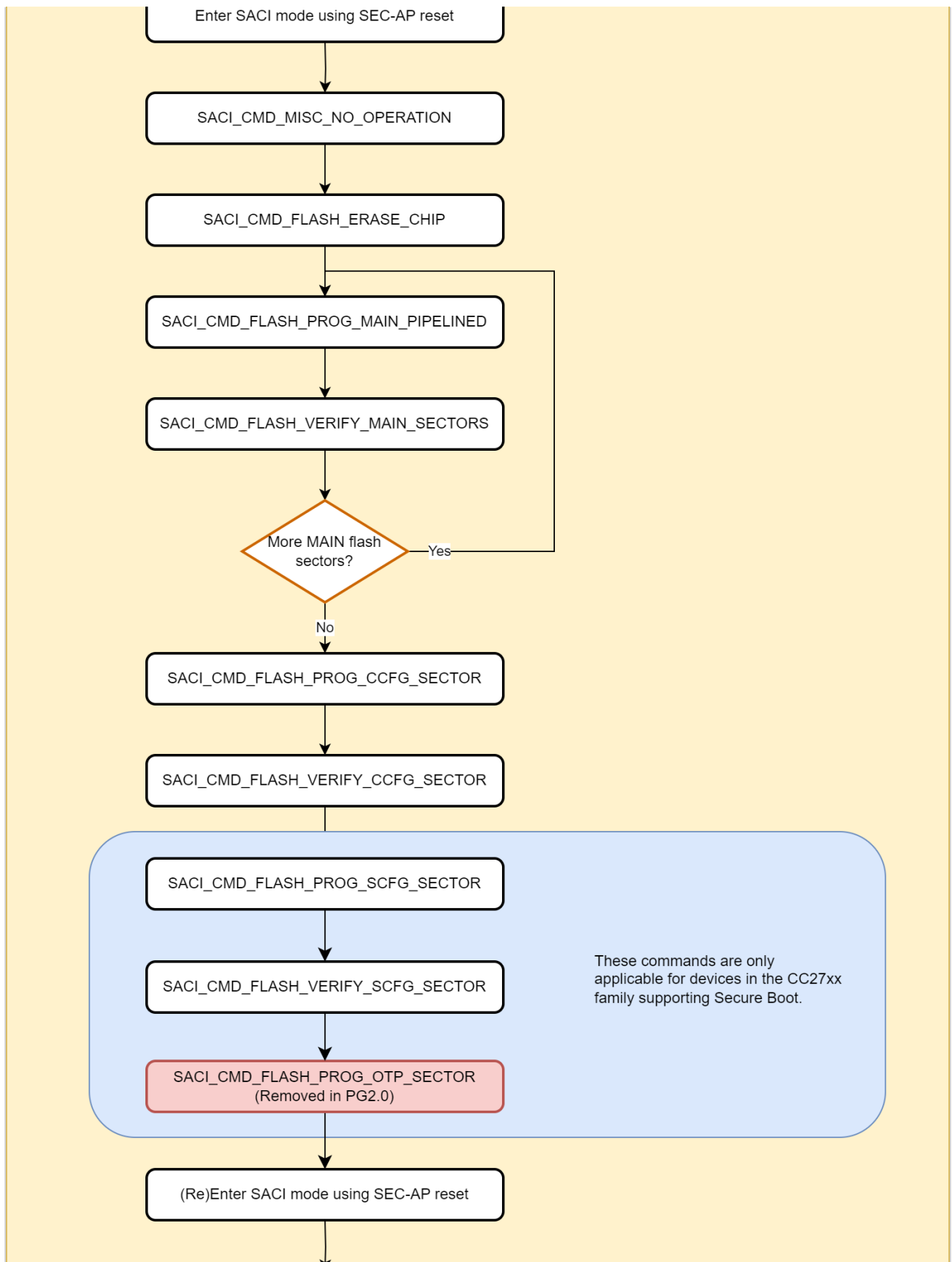
Sector	Base address	Length
CCFG	0x4E020000	2048
OTP (Removed in PG2.0)	0x4E020800	2048
SCFG	0x4E040000	1024

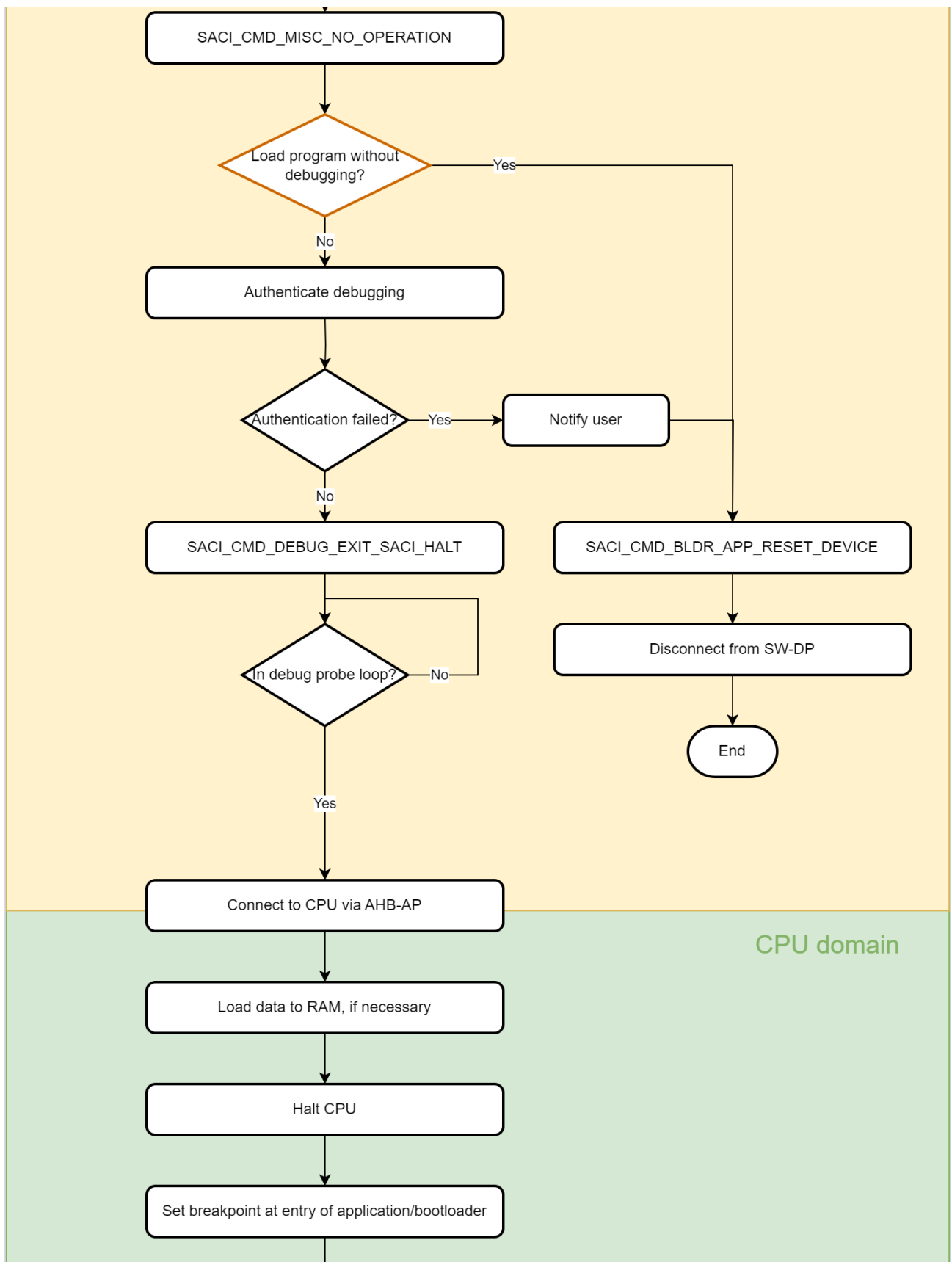
7.2 Suggested flash programming sequence

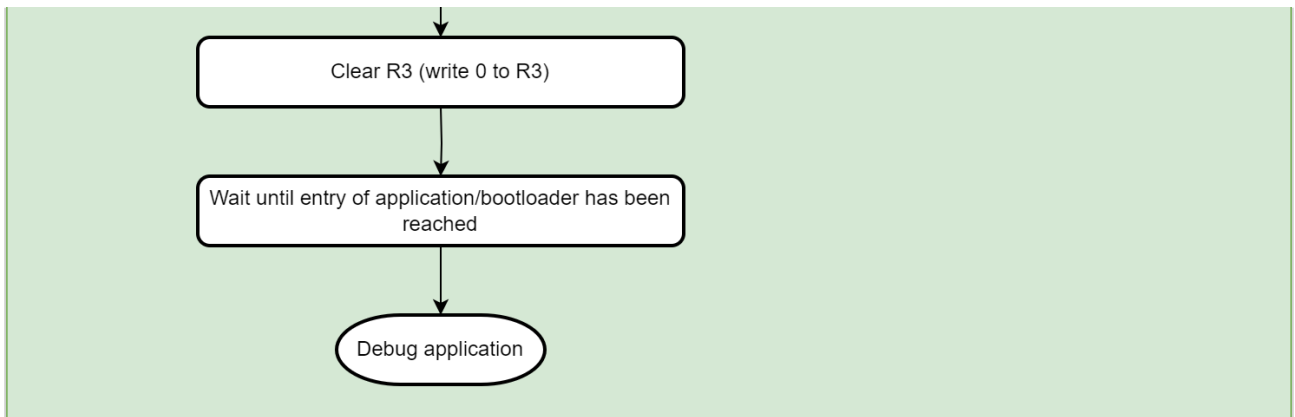
Below flow chart is the suggested way to program flash.

- i** The flow chart is divided into the following "domains":
- AP domain: Direct AP register access only. Does not require access to AHB-AP/CPU.
 - CPU domain: CPU access through AHB-AP. Require access to AHB-AP/CPU. Does not require access to other AP's than AHB-AP.





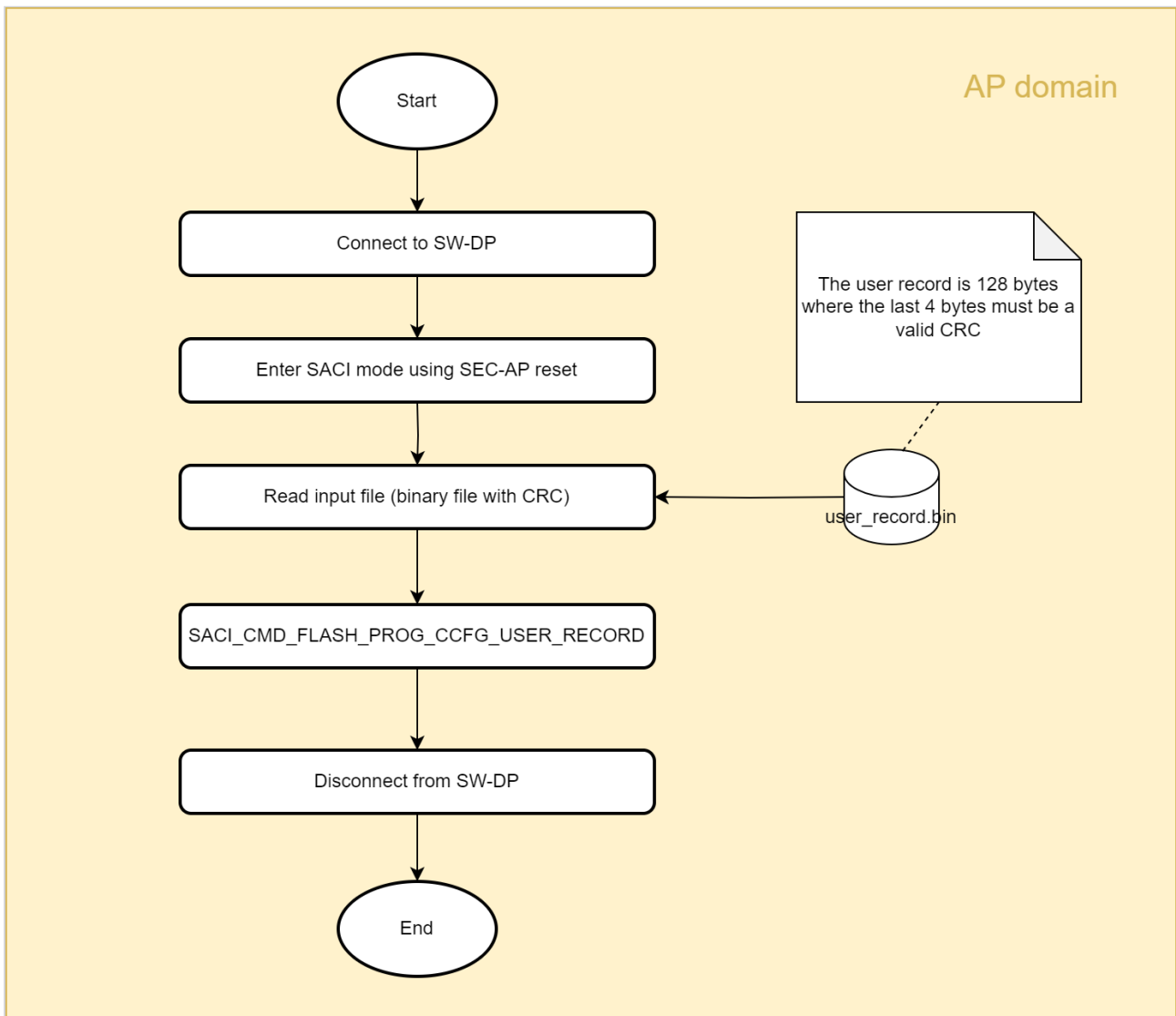




8 Programming of CCFG User record

Below is a flow chart with a suggested way to program the CCFG User record in a separate step during the production process.

See [Technical Reference Manual \(TRM\)](#)⁹ for more details

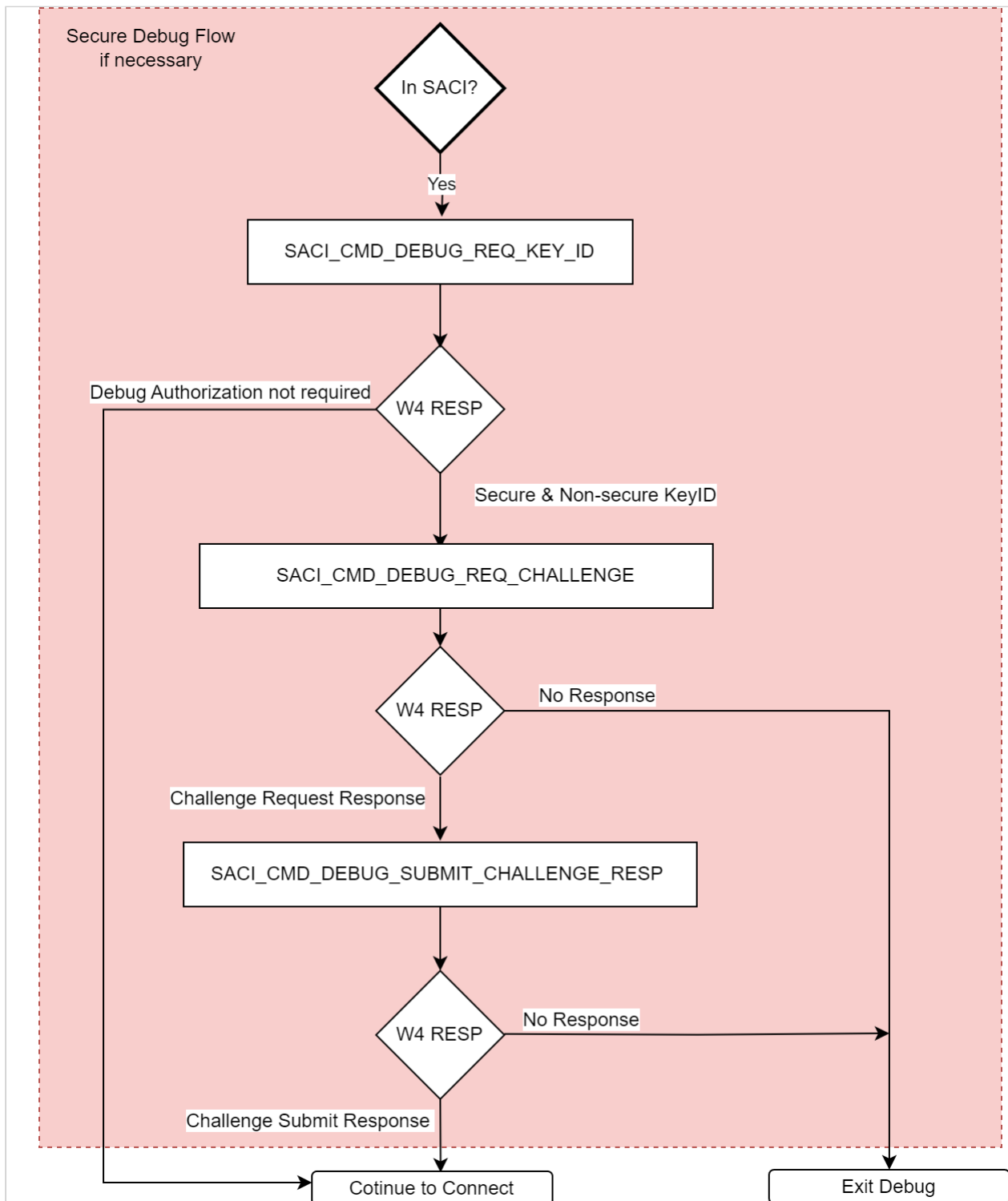


⁹ <https://www.ti.com/lit/pdf/swcu193>

9 Secure Debug Commands Sequence

Below is a flow chart with a suggested flow of secure debug commands for debug authorization in **CC27xx PG2.0**.

In section 7(Connect to target) compared to password authentication same flow can be replaced with Secure Debug Flow



10 Use cases

10.1 Download and debug

10.1.1 Use Case

This is the "standard" way to debug an application. The user wants to download the application and start debugging it.

10.1.2 UI

A dedicated button is suggested for this.

10.1.3 Sequence

1. Download program.
 - a. See [Suggested flash programming sequence \(see page 33\)](#)
 - b. Follow the 'No' branch of the "Load program without debugging" decision block.
2. Start debugging (Unless authentication failed)

10.2 Download without debugging

10.2.1 Use case

If the user only wants to download the application, but not debugging it, this is the use case to use. This can be used by dedicated programming tools.

10.2.2 UI

A dedicated button is suggested for this.

10.2.3 Sequence

1. Download program.
 - a. See [Suggested flash programming sequence \(see page 33\)](#)
 - b. Follow the 'Yes' branch of the "Load program without debugging" decision block.

10.3 Debug without downloading

10.3.1 Use case

If the target already contains the desired application, there is no need to download the application, and it may not be desirable to re-download the application, if for example the application has modified the flash.

10.3.2 UI

A dedicated button is suggested for this.

10.3.3 Sequence

1. Connect to CPU
 - a. See [Connect to target \(see page 24\)](#)
 - b. Follow the 'Yes' branch of the "Reset when connecting to CPU?" decision block.
 - c. One of the desired breakpoints in the "Set desired breakpoints" block should be at the entry of the application, at main or similar.

10.4 Attach to running target

10.4.1 Use case

- Diagnostics in the field
- During development: Connecting to specific nodes in a large system to extract information or diagnose problems.
- During development: "power debugging". Running code without a debug connection to get real power profile most of the time and only connecting a debugger at later point or if a problem arises.



It is only possible to connect to a running target if it does not require authentication. If it requires authentication, the device must be reset to enter SACI mode and provide the authentication.

10.4.2 UI

A dedicated button is suggested for this.

10.4.3 Sequence

1. Connect to CPU
 - a. See [Connect to target \(see page 24\)](#)
 - b. Follow the 'No' branch of the "Reset when connecting to CPU?" decision block.
 - c. If AHB-AP is not enabled (if for example debug authentication is required), give a warning or abort connecting with appropriate error message.

10.5 Debug through STANDBY

10.5.1 Use case

Most user applications will often enter STANDBY and it will therefore be useful to be able to debug such an application, otherwise it will not be able to debug most applications.

10.5.2 UI

The IDE should indicate to the user if the device is in FAKESTANDBY (or STANDBY)

10.5.3 Sequence

If the debugger issues the system power up request when connecting to the target, which it must do to connect to the CPU, the debugger does not need to do anything to be able to debug the application.

When the application issues a command to enter STANDBY, the device will enter FAKE_STANDBY instead, due to the system power up request.

The debugger should periodically pull CFGAP:DEVICESTATUS, and if the device is in FAKESTANDBY (or STANDBY) it should indicate this to the user.

10.6 Debug through reset

10.6.1 Use case

A device can be reset for many different reasons.

For CC23xx/CC27xx devices, the debugger needs to reconnect to the SW-DP after a reset, so the debugger must handle this to create the illusion of maintaining an open debug connection.

If the device is reset regularly, it would be desirable for the debugger to be able to recover from the reset, without the user have to manually reconnect on after every reset.

In some scenarios manually reconnecting to the target is not sufficient to accomplish the goal of the debug session. If the user wants to debug the handling of the different reset reasons, it should be possible to

reconnect before the reset handling happens. Furthermore, if the application requires authentication, it is necessary to provide that authentication when the boot code enters SACL mode after the original reset. The debugger cannot issue its own reset to enter SACL mode, because then the application will not handle the original reset, it will handle the reset caused by the debugger.

10.6.2 UI

Recommended solution:

After a reset there should be some prominent indication that this has happened in the UI (red status bar icon, red warning text in log window, warning text in register view or similar).

Other possible solutions:

- Pop-up dialog where the user confirms a debug reconnection attempt.
 - One problem with this solution is that the debugger must reconnect to the device with the SACL inactivity timeout, to send a SACL command to stay in SACL. If the user has to confirm reconnecting to the device, then the SACL inactivity timeout will most likely expire before the user confirms reconnecting to the device.

10.6.3 Sequence

1. Debugger must periodically pull CFGAP:DEVICESTATUS:BOOTSTA.
2. When the device is reset, the reading of CFGAP:DEVICESTATUS:BOOTSTA or any other SWD transaction will fail.
3. The debugger must detect this and Reconnect to the SW-DP
 - a. See [Connect to target \(see page 24\)](#)
 - b. Follow the 'Yes' branch of the "Reconnecting after reset?" decision block.
 - c. In the "Set desired breakpoints" block, the previously configured breakpoints (from before the reset) must be reconfigured.

10.7 Debug through SHUTDOWN

10.7.1 Use case

Ability to debug an application that relies on SHUTDOWN mode to achieve the desired power consumption (TPMS for instance).

10.7.2 UI

(TODO: Should there be any UI, for example a pop-up dialog, to indicate that a shutdown wakeup has happened? Also when the wakeup is due to SWD activity?)

The IDE should indicate to the user if the device is in SHUTDOWN

10.7.3 Sequence

1. Debugger most periodically pull CFGAP:DEVICESTATUS:BOOTSTA.
2. When the device is reset (due to SHUTDOWN wakeup), the reading of CFGAP:DEVICESTATUS:BOOTSTA or any other SWD transaction will fail.
3. The debugger must detect this and Reconnect to the SW-DP
 - a. See [Connect to target \(see page 24\)](#)
 - b. Follow the 'Yes' branch of the "Reconnecting after reset?" decision block.
 - c. In the "Set desired breakpoints" block, the previously configured breakpoints (from before the shutdown entry) must be reconfigured.
 - d. If returning to shutdown mode, the debugger should indicate that the device is in SHUTDOWN mode.

10.8 Debugging without system power up request

10.8.1 Use case

It can be useful to debug a target without issuing a system power up request. CC23xx/CC27xx devices does still support some features when system power up request has not been issued.

When debugging without using system power up request, the power profile of the target will be closer to the real world power profile. The target will for example enter real STANDBY instead of FAKE_STANDBY. The timing will also resemble the real world timing better. The device will for example enter and exit STANDBY as it would in the real world.

Note that SWD activity will still wakeup the device from SHUTDOWN, so the power profile and timing will be altered if the device is going in to SHUTDOWN mode.

When debugging without system power up request, it is not possible to connect to the CPU, it is only possible to connect to the SW-DP.

Possible ways to debug the application when only connected to the SW-DP (not connected to the CPU)

- Memory access
- Using DEVICESTATUS:BOOTSTA from CFG-AP to monitor the status of the application
- Using the SEC-AP mailbox to communicate with the application
- Using ET-AP to extract energy trace information.

10.8.2 UI

TBD

10.8.3 Sequence

1. Connect to SW-DP without system power up request
 - a. See [Connect to target \(see page 24\)](#)
 - b. Use the 'No' branch of the "Some condition" decision block.
2. Debug

10.9 Debugging a device with SWDCK/SWDIO pins driven by application

10.9.1 Use case

If for example the application on the target is driving at least one of the SWD pins (SWDCK and SWDIO), the debug probe will have no control of the SWD pins and it will not be able to connect to the target.

To connect to such a target, the pin-reset must be asserted while sending the DORMANT to SWD sequence.

It is not desirable to use this connection method if it is not necessary, since it will require an extra physical line to the target (the reset line), and it will not be possible to non-intrusively connect to the target. Since the reset is intrusive.

This use case applies to all the other use cases.

10.9.2 UI

To connect to a device with SWD pins configured as outputs, there should be an option (probably a checkbox) in the IDE to enable below sequence. Otherwise the normal connect sequence should be used.

It is not a requirement to be able to change this option during a debug session.

Suggestions for name of the option:

- Reset when connecting
 - Pro: This clearly informs the user what happens when the option is enabled.
 - Con: This does not inform the user of when to use this option
- Application uses SWD pins (or something similar)
 - Pro: The user knows when to use the option
 - Con: The user does not know what the option does.

10.9.3 Sequence

If the option to use pin-reset is selected by the user, the debugger must use the 'Yes' branch of the "Connect using pin reset?" decision block in the flowchart in [Connect to target \(see page 24\)](#), whenever the debugger is connecting to the target.

If the option to use pin-reset is not selected by the user, the debugger must use the 'No' branch of the "Connect using pin reset?" decision block in the flowchart in [Connect to target \(see page 24\)](#), whenever the debugger is connecting to the target.

10.10 Communicate with application through SEC-AP mailbox

10.10.1 Use case

It is often useful for the user to provide a text interface to the device. This is often done by utilizing an UART port on the device.

For CC23xx/CC27xx devices, the SEC-AP mailbox can be used for such a purpose. This will allow the user to save an UART port.

10.10.2 UI

The IDE should contain a terminal window or similar for the SEC-AP mailbox.

10.10.3 Sequence

The way to handle this is not exactly clear yet, this is one suggested solution:

Receive data:

1. The debugger must continuously poll the SECAP:RXCTL
 - a. The polling frequency can maybe depend on the time since the last word was received (and maybe also time since the last word sent).
 - b. If SECAP:RXCTL indicates that a word is ready, read it from SECAP:RXD and print the data to the console window.

Send data (provided by user through terminal window):

1. The debugger must continuously poll the SECAP:TXCTL
 - a. If SECAP:TXCTL indicates that there is no pending data, send one word of the data to be send to SECAP:TXD

11 Appendix

11.1 SACL commands for CC27xx devices

11.1.1 Program SCFG Sector - **SACL_CMD_FLASH_PROG_SCFG_SECTOR**

Program the SCFG sector.

11.1.1.1 Restrictions

Since SCFG is a 2T sector all bytes in the SCFG will be random after erasure. For this reason, the SCFG can only be programmed if the SCFG is invalid. SCFG is invalid if the boot code detects an issue in the CRC field or if the SCFG was specifically erased during the SACL Idle loop. The only way to do this is the SACL_CMD_FLASH_ERASE_CHIP cmd which will invalidate the SCFG.

11.1.1.2 Side-Effects

This command modifies SCFG. Certain commands are not allowed after SCFG sector programming:

- SACL_CMD_DEBUG_EXIT_SACL_HALT
- SACL_CMD_BLAPP_EXIT_SACL_RUN

Reset the device and reenter SACL to reenable those commands.

11.1.1.3 Parameter Words

Word s	Bits	Field	Value	Description
0	7:0	cmdId	0x1A	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	byteCount	-	Number of bytes to program. If 0, the command has no effect. • min = 276 bytes = (SCFG_SIZE - (sizeof(keyEntry) * 17)) • max = 1024 bytes = SCFG_SIZE

Words	Bits	Field	Value	Description
1	31:0	key	0xB7E3A08F	Flash operation key (magic number)
257:2	31:0	data	User defined	Bytes to be programmed: The first byte is in bits 7:0 of word 2

11.1.1.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1A	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result. One of the common results, or: - INVALID_KEY_PARAM if the key parameter is incorrect - FLASH_FSM_ERROR if the flash hardware FSM reported an error - NOT_ALLOWED if restrictions are not met (the command has no effect)
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.1.5 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED
- If byteCount is invalid: Fail with result
- If key is invalid: Fail with result INVALID_KEY_PARAM

11.1.2 Verify Flash SCFG Sector - SACL_CMD_FLASH_VERIFY_SCFG_SECTOR

Verify the specified parts of the flash SCFG sector, using CRC32. There is no check for blank SCFG because it is in 2T memory which will read as random when unprogrammed.

The SCFG is currently only covered by a single CRC which spans from the first field to the last.

- SCFG
 - Data:
 - Scfg.hsmCfg.publicKeyHash through entire Scfg.policyCfg
 - Embedded CRC32:
 - Scfg.crc32

The command perform one of these checks:

- Compute and check CRC32 of data, using only embedded CRCs
- Compute and check CRC32 of data, using both embedded CRCs and expected CRCs (provided in the command)

The command result is SUCCESS if all performed checks are successful.

11.1.2.1 Restrictions

These conditions must be true:

- Fcfg.permissions.allowFlashVerify = 0xA (FCFG_PERMISSION_ALLOW)
- If CCFG is valid:
 - Ccfg.permissions.allowFlashVerify= 0xA (CCFG_PERMISSION_ALLOW)
- **For CC27xx PG2.0** - If SCFG is valid:
 - Scfg.permissions.allowFlashVerify = 0xA (SCFG_PERMISSION_ALLOW)

11.1.2.2 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1B	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header

Words	Bits	Field	Value	Description
	16	checkExpCrcs	-	Check SCFG data also against the expected CRC32 fields the command: 0: Use only embedded CRCs 1: Use both embedded CRCs and the expected CRCs
	31:17	reserved0	0x00	Reserved
1	31:0	expCrc32	-	Expected CRC32 of the SCFG: - If checkExpCrcs = 1: Same value as in Ccfg.bootCfg.crc32 - Otherwise: Don't care

11.1.2.3 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1B	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result. One of the common results, or: - CRC32_MISMATCH if any of the performed CRC checks failed - NOT_ALLOWED if restrictions are not met
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.2.4 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED

11.1.3 Program OTP Sector - **SACI_CMD_FLASH_PROG_OTP_SECTOR (REMOVED in PG2.0)**

Program the One Time Programmable(OTP) sector. The OTP sector may not function exactly as you would expect given the OTP name. In reality, the OTP sector is simply another 1T nonmain sector just as the CCFG is. It has the same physical properties as the CCFG which means that the OTP sector can in fact be physically written multiple times. OTP is used to hold HSM assets, so from the application's perspective it is designed to be immutable. This command can only be used if it is preceded by a SACI_CMD_FLASH_ERASE_CHIP resulting in the OTP sector being completely erased.

11.1.3.1 Availability

This command is only available for the **CC27xx PG1.x** device.

11.1.3.2 Restrictions

All bytes in the OTP sector must be 0xFF before the OTP sector programming begins (a chip erase must have been performed since the previous OTP programming).

11.1.3.3 Side-Effects

This command modifies OTP. Certain commands are not allowed after OTP sector programming:

- SACI_CMD_DEBUG_EXIT_SACI_HALT
- SACI_CMD_BLAPP_EXIT_SACI_RUN

Reset the device and reenter SACI to reenable those commands.

11.1.3.4 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x19	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	byteCount	-	Number of bytes to program. If 0, the command has no effect.

Words	Bits	Field	Value	Description
1	31:0	key	0xB7E3A08F	Flash operation key (magic number)
513:2	31:0	data	User defined	Bytes to be programmed: - The first byte is in bits 7:0 of word 2 - Although only the first <code>byteCount</code> bytes will be written, zero-padding must be added to send the full size of the OTP sector

11.1.3.5 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x19	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result. One of the common results, or: - INVALID_KEY_PARAM if the key parameter is incorrect - FLASH_FSM_ERROR if the flash hardware FSM reported an error - NOT_ALLOWED if restrictions are not met (the command has no effect)
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.3.6 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED
- If key is invalid: Fail with result INVALID_KEY_PARAM

11.1.4 Read Secure Boot or HSM FW update Status - **SACI_CMD_GET_SECBOOT_HSMFWUPDATE_STATUS**

It may be useful when a host needs to determine why a device may not be booting into the application. The command is used to read the STATUS field from the PMCTL_O_AONSTA0[14:12].

11.1.4.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

11.1.4.2 Restrictions

No restrictions

11.1.4.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x23	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved

11.1.4.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x23	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	SCR_SUCCESS
	31:24	dataWordCount	1	Size of additional response data, in number of 32-bit words
1	31:0	status	-	SB_STA_NA (value 0) : Regular boot w/o a new image, No error. SB_STA_UPDATE_SUCCESS(value:1) : Requested update was successful (any types) SB_STA_FIH_ERROR(value: 2) : ROM panic mode due to a fault Injection SB_STA_IMG_REJECTED (value:3) : Invalid image detected (any types). SB_STA_IMG_VERIF_FAILED (value:4) : The target image failed to verify (anytypes) SB_STA_UPDATE_FAILED(value:5) : Requested image update process failed. SB_STA_INVALID_CONFIG(value6) : SCFG configuration is invalid SB_STA_HSM_BOOT_FAILURE(value:7) : HSM FW failed to boot

11.1.5 Read the HSM system information - **SACI_CMD_HSM_GET_SYS_INFO**

This command returns the HSM system information.

11.1.5.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

11.1.5.2 Restrictions

No restrictions

11.1.5.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x22	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved

11.1.5.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x22	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	SCR_HSM_BOOT_FAILED: HSM failed to boot SCR_SUCCESS: HSM system info has been retrieved successfully
	31:24	dataWordCount	9	Size of additional response data, in number of 32-bit words
1	7:0			HSM FW Version Patch

Words	Bits	Field	Value	Description
	15:8			HSM FW Version Minor
	23:16			HSM FW Version Major
	31:24			HSM FW Rollback ID
2	7:0			HSM HW Version Patch
	15:8			HSM HW Version Minor
	23:16			HSM HW Version Major
	31:24			Reserved
3	15:0			Available Memory Size
	18:16			HOST ID of the Token Writer
	19			1: Host is in non Secure side 0: Host is in Secure side
	26:20			Reserved
	27			1: COID is set 0: COID is not set

Words	Bits	Field	Value	Description
	31:28			Mode 0000: 'Functional Mode' state, the Power-On Self-Tests have been executed successfully 1111: 'Functional-Mode' state, with successful Login done 0011: 'Error' state, Login failed 0100: 'Error' state, at least one of the self-tests has failed 0101: 'Error' state, DMA error detected 0110: 'Error' state, NRBG or DRBG stuck error detected 0111: 'Error' state, key generation failed
4	31:0			Identify of the caller
5	11:0			Reserved
5	15:12			OTP Anomaly 0000: No anomaly 0001 : OTP area is empty 0010 : Unsupported size 0100 : FAT Error 1000: OTP are is zeroized
5	31:16			SelftestActive The value of this field is only applicable when the system is in 'Error' state A non-zero value refers to the self-test that caused the transition to the 'Error' state.
6	7:0			HSM BOOT FW Version Patch
	15:8			HSM BOOT FW Version Minor
	23:16			HSM BOOT FW Version Major
	31:24			Reserved

Words	Bits	Field	Value	Description
7	7:0			Custom BOOT FW Version Patch
	15:8			Custom BOOT FW Version Minor
	23:16			Custom BOOT FW Version Major
	31:24			Reserved
8	7:0			Custom FW Version Patch
	15:8			Custom FW Version Minor
	23:16			Custom FW Version Major
	31:24			Reserved
9	7:0			Custom HW Version Patch
	15:8			Custom HW Version Minor
	23:16			Custom HW Version Major
	31:24			Reserved

Note:

Flashloader shall display Custom FW Version as **HSM FW Version** to users.

11.1.6 Request Debug Key ID - **SACL_CMD_DEBUG_REQ_KEY_ID**

Request key ID for debug authentication

This command must be used to check whether debugging is allowed, and obtain the key ID for SACL_CMD_DEBUG_SUBMIT_CHALLENGE_RESP, if required. As indicated, there are three possible states:

Ccfg.debugCfg.authorization	Description	Indicated by
0xA5	Debug authentication is required	Response contains a 64-bit debug key ID based on the requested auth level.
0x5A	Debug authentication is not required	Response contains no key ID. Debugging will be enabled after exiting SACI.
0xC3	Only non-invasive debugging is allowed	Response contains no key ID. Debugging will be enabled after exiting SACI.
Any other value	Debugging is not allowed	Response result is NOT_ALLOWED

The key ID is fetched from Scfg.debugAuthCfg.secureKey or Scfg.debugAuthCfg.nonSecureKey based on the requested auth level.

As described in the SACI_CMD_DEBUG_REQ_CHALLENGE and SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP sections. Once the debug auth process has started it must be completed without interruption by non debug auth related commands. For this purpose, the SACI_CMD_DEBUG_REQ_KEY_ID is considered a debug auth related command meaning an ongoing debug auth process will not be halted by submitting this command. On the other hand, the debug auth process will also not begin by submitting this command meaning it can be submitted at any time without consequence on the state of a future or ongoing debug auth process.

11.1.6.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

This command is only available for **CC27xx PG2.0** only.

11.1.6.2 Restrictions

CCFG must be valid.

Ccfg.debugCfg.authorization = 0xA5, 0x5A or 0xC3.

11.1.6.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1D	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved
1	31:0	authLevel	User defined	The requested authentication level for specifying which key ID to return • 0x401AA5A5 for requesting the key ID capable for secure debug access • 0x989B5A5A for requesting the key ID capable for non-secure debug access • 0xFB86C3C3 for requesting the key ID capable for non-invasive only debug access

11.1.6.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1D	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result. One of the common results, or: • INVALID_DEBUG_AUTH_LVL_PARAM if the authLevel is invalid • NOT_ALLOWED if restrictions are not met

Words	Bits	Field	Value	Description
	31:24	dataWordCount	0 or 2	Size of additional response data, in number of 32-bit words: • 2 if debug authentication is required • 0 if debug authentication is not required, or debugging is not allowed
2:1	31:0	keyId	Scfg.debugAuthCfg.secureKey.keyID or Scfg.debugAuthCfg.nonSecureKey.keyID	The 64-bit debug key ID: • The first byte of the key ID is in bits 7:0 of word 1 • The last byte of the key ID is in bits 31:24 of word 2

11.1.6.5 Check Sequence

These checks are performed before command execution, in the indicated order:

- If Ccfg is valid and Scfg is valid and Ccfg.debugCfg.authorization = 0xA5 but requested authLevel is invalid: Fail with result INVALID_DEBUG_AUTH_LVL_PARAM
- If any restriction is violated: Fail with result NOT_ALLOWED

11.1.7 Request Debug Challenge - **SACI_CMD_DEBUG_REQ_CHALLENGE**

If the SACI_CMD_DEBUG_REQ_KEY_ID command returns a 64-bit key ID, the host must use this command to request the 40 byte debug challenge vector.

The content of the challenge is configured via the Scfg.debugAuthCfg.challengeVector.

- If Scfg.debugAuthCfg.challengeVector.lifetime = 0xF1A1A5A5 (SCFG_DBGAUTH_EPHEMERAL_LIFETIME)
 - The challenge vector will contain a cryptographically random number causing no two requested vectors to be identical
- If Scfg.debugAuthCfg.challengeVector.lifetime = 0x51445A5A (SCFG_DBGAUTH_ENDLESS_LIFETIME)
 - The challenge vector will **not** contain a cryptographically random number causing every requested vector for a specific device to be identical

- If `Scfg.debugAuthCfg.challengeVector.deviceConst= 0x3262A5A5`
(`SCFG_DBGAUTH_DEVICE_MAC_CONST`)
 - The challenge vector will contain the device specific MAC address causing no two devices to produce the same challenge vector
- If `Scfg.debugAuthCfg.challengeVector.deviceConst = 0x62BB5A5A`
(`SCFG_DBGAUTH_DEVICE_ZERO_CONST`)
 - The challenge vector will **not** contain the device specific MAC address causing every requested vector from device to device to produce the same vector (depending on the value of `Scfg.debugAuthCfg.challengeVector.lifetime`)

The requested 40 byte debug challenge vector must then be signed by the host using the private key which corresponds to the public key on the device. The exact public/private key pair to use is determined by the 64-bit key ID returned by the `SAC_CMD_DEBUG_REQ_KEY_ID` command. The type of cryptographic signing depends on the type of the key pair being used. The type of key pair depends on the configuration of `Scfg.secBootCfg.policyCfg.authAlgorithm`

After signing, the host must continue the debug auth process by sending the `SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP` command.

This command will trigger the start of the "debug auth process". This process shall not be interrupted by non debug auth related commands. If it is, then the process will be halted and this command must be submitted again to complete the debug auth process. The list of debug auth related commands that can be sent without halting the ongoing debug auth process are:

- `SACI_CMD_DEBUG_REQ_KEY_ID`
- `SACI_CMD_DEBUG_REQ_CHALLENGE`
 - Note that if this command is submitted again, it will technically begin a new debug auth process which could result in a different challenge vector based on the configuration of `Scfg.debugAuthCfg.challengeVector`
- `SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP`

11.1.7.1 Availability

This command is not available in the `P1STBDAY` lifecycle state (result is `INVALID_CMD_ID`).

This command is only available for **CC27xx PG2.0**.

11.1.7.2 Restrictions

CCFG must be valid.

SCFG must be valid.

`Ccfg.debugCfg.authorization = 0xA5 or 0x5A`.

11.1.7.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1E	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved
1	31:0	authLevel	User defined	The requested authentication level for the debug authentication process being started • 0x401AA5A5 for requesting a challenge vector capable for secure debug access • 0x989B5A5A for requesting a challenge vector capable for non-secure debug access • 0xFB86C3C3 for requesting a challenge vector capable for non-invasive only debug access

11.1.7.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1E	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word

Words	Bits	Field	Value	Description
	23:16	result	-	Command result. One of the common results, or: - NOT_ALLOWED if restrictions are not met - HSM_BOOT_FAILED if the HSM FW was unable to boot while generating a cryptographically random number - CMD_FAILED if the HSM FW was unable to generate a cryptographically random number - INVALID_DBG_AUTH_CONFIG if an invalid Scfg.debugAuthCfg was detected - INVALID_DBG_AUTH_LVL_PARAM if authLevel was invalid
	31:24	dataWordCount	0 or 10	Size of additional response data, in number of 32-bit words
41:1	31:0	challengeVector	-	The requested 40 byte debug challenge vector

11.1.7.5 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED

11.1.8 Submit Debug Challenge Response - SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP

If the SACI_CMD_DEBUG_REQ_KEY_ID command returns a 64-bit key ID, the host must first request a challenge via the SACI_CMD_DEBUG_REQ_CHALLENGE command, sign the challenge and then use this command to submit the challenge response.

This command will trigger the end of the "debug auth process". This process shall not be interrupted by non debug auth related commands. If it is, then the process will be halted and the debug auth process must be started again. The list of debug auth related commands that can be sent without halting the ongoing debug auth process are:

- SACI_CMD_DEBUG_REQ_KEY_ID
- SACI_CMD_DEBUG_REQ_CHALLENGE
 - Note that if this command is submitted again, it will technically begin a new debug auth process which could result in a different challenge vector based on the configuration of Scfg.debugAuthCfg.challengeVector
- SACI_CMD_DEBUG_SUBMIT_CHALLENGE_RESP

- Note that this command will end the ongoing debug auth process

Take special care to understand the CC27xx PG2.0's persistent debug feature

After the SAC_CMD_DEBUG_SUBMIT_CHALLENGE_RESP command is completed successfully, the CC27xx PG2.0 device will persist the debug authentication. This debug authentication will persist either until the device goes through a power cycle or until the SACI_CMD_DEBUG_CLOSE_SESSION command is sent.

11.1.8.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

This command is only available for **CC27xx PG2.0**.

11.1.8.2 Restrictions

CCFG must be valid.

SCFG must be valid.

There must be an ongoing debug auth process by first submitting the SACI_CMD_DEBUG_REQ_CHALLENGE command.

Ccfg.debugCfg.authorization = 0xA5 or 0x5A.

11.1.8.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1F	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	wordCount	-	Length of the remainder of the command in 32-bit words
1	31:0	pubKeyByteCount	-	Length of the public key in 8-bit bytes (max = 512)
2	31:0	signatureByteCount	-	Length of the signature in 8-bit bytes (max = 384)

Words	Bits	Field	Value	Description
131:3	31:0	publicKey	-	The public key for debug authentication in the DER format. Length of the key depends on the type of key and signing process. • The first byte of the public key is in bits 7:0 of word 3 • If the length of the key is less than the max then padding bytes of 0x00 should be added to fill up the entire field
516:132	31:0	signature	-	The resulting signature from signing the challenge vector Length of the signature depends on the type of key and signing process. • The first byte of the signature is in bits 7:0 of word 132 • If the length of the signature is less than the max then padding bytes of 0x00 should be added to fill up the entire field

11.1.8.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x1F	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word

Words	Bits	Field	Value	Description
	23:16	result	-	Command result. • NOT_ALLOWED if restrictions are not met • INVALID_SIZE_PARAM if either pubKeyByteCount or signatureByteCount are invalid • KEY_HASH_MISMATCH if the public key does not match the appropriate publicKeyHash in Scfg.debugAuthCfg • INVALID_DBG_AUTH_LVL_PARAM if the internal state of the auth level of the ongoing debug auth process is incorrect • CMD_FAILED if SHA256 hash calculation failed • INVALID_KEY_PARAM if the public key provided cannot be parsed correctly • CHALLENGE_RESP_VERIFY_FAIL if the signature cannot be verified correctly • INVALID_DBG_AUTH_CONFIG if the Scfg.secBootCfg.policyCfg.authAlgorithm is not a valid algorithm • SUCCESS if the debug authorization process was successful
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.8.5 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED
- If the pubKeyByteCount or signatureByteCount are invalid: Fail with result INVALID_SIZE_PARAM
- if the signature does not pass verification: Fail with the result of CMD_FAILED

11.1.9 Submit Debug Challenge Response - **SACI_CMD_DEBUG_CLOSE_SESSION**

The CC27xx PG2.0 device has a feature known as debug persistence. Once the SACI_CMD_DEBUG_SBUMIT_CHALLENGE_RESP command has returned with success the device will persist the successful debug authentication. This means that a simple reset of the device will not clear the debug authentication status. There are three ways to clear the debug persistence.

1. Sending the SACI_CMD_DEBUG_CLOSE_SESSION command
2. Power cycling the device

- Using either the debugger and or a flash application to clear the debug persistence bits in the relevant SYS0 register and then performing a SW reset.

11.1.9.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

This command is only available for **CC27xx PG2.0**.

11.1.9.2 Restrictions

No restrictions

11.1.9.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x20	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved

11.1.9.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x20	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	SUCCESS	Command result

Words	Bits	Field	Value	Description
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.9.5 Check Sequence

No checks

11.1.10 HSM FW Provision- **SACI_CMD_HSM_FW_PROVISION**

This allows the HSM provision process to be triggered and completed with only one SACI command. For this command, SACI has a data buffer to store the maximum HSM FW Update Image (HUI) size of 128kB.

11.1.10.1 Availability

This command is only available for the **CC27xx PG2.0** device only.

11.1.10.2 Flow Control Mechanism and Response Handling

When SACI receives the initial command it will first validate the input and then prepare the internal data buffer to be filled with the new HSM FW. This process is relatively lengthy and requires synchronization with the host. The host shall not send subsequent HUI words until SACI is done preparing the buffer. To synchronize, the host shall poll on the DEBUGSS:RXCTL register until the CMD_WORKING bit has been cleared. When the host sees the CMD_WORKING bit has been cleared it can continue by sending back to back HUI data words until the entirety of the HUI has been sent. Note that SACI will once again set the CMD_WORKING bit to indicate that it once again receiving and processing HUI words. [See also Chapter 8.3.1.4 in the Technical Reference Manual](#) [Technical Reference Manual \(TRM\)](#)¹⁰.

When SACI receives the last of the HUI words, this ends the transfer and begins the authentication/verification.

If SACI detects that the huiSize is larger than the maximum of 128kB it will respond with INVALID_SIZE_PARAM. If SACI detects an authentication failure of the HSM FW it will respond with HSM_UPDATE_FAILED.

11.1.10.3 Other considerations

When this command is used on a device which does not yet have it's SCFG programmed (empty/blank device) the result will always HSM_FW_CRYPTO_FAIL. This is due to the optional customer signature verification which is performed. If the SCFG is un-programmed, the ROM is required to assume customer

¹⁰ <https://www.ti.com/lit/pdf/swcu193>

signature verification must take place which will result in failure. One must first program the device (SCFG included) and properly configure the SCFG.hsmUpdateKeys field prior submitting this command.

11.1.10.4 Restrictions

If any of the following commands have been sent during the current SACI session. The device must be reset to enter a new SACI session prior to executing this command. The provision command will fail otherwise

- SACI_CMD_FLASH_ERASE_MAIN_APP
- SACI_CMD_FLASH_ERASE_CHIP
- SACI_CMD_HSM_GET_SYS_INFO
- SACI_CMD_DEBUG_REQ_CHALLENGE

11.1.10.5 Parameter Words

Initial/first command

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x21	Command ID
	15:8	respSeqNumber	User defined	Base response sequence number
	31:16	reserved0	0x0000	Reserved
1	31:0	huiSize	-	Total length, in words, of the HUI data which must be transferred to the device

Then, for each HUI word:

Words	Bits	Field	Value	Description
0	31:0	huiData	User defined	HUI Data: • The first byte of the is in bits 7:0 of word 0

11.1.10.6 Response Words

This response is generated either immediately due to invalid huiSize or after the last HUI word is received and authentication/verification is completed:

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x21	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Response sequence number. The response sequence number shall be the initial respSeqNumber from the input command +1 for each HUI data word sent.
	23:16	result	-	Command result. One of the common results, or: - SCR_INVALID_SIZE_PARAM - the HUI size parameter is invalid - SCR_SUCCESS - HSM FW update finished successfully - SCR_FLASH_FSM_ERROR - HSM FW update failed - SCR_HSM_BOOT_FAILED - HSM FW has been updated but HSM failed to boot - SCR_HSM_FW_CRYPTOFail - HSM FW image failed to be verified - SCR_HSM_FW_HDR_INVALID - HSM FW image type is incorrect - SCR_HSM_FW_VER_INVALID - Incorrect HSM FW version - SCR_CMD_FAILED - Default
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.1.10.7 Checks

These checks are performed before command execution, in the indicated order:

- If huiSize is invalid: Fail with result INVALID_SIZE_PARAM
- If any restriction is violated: Fail with result NOT_ALLOWED

These checks are performed after the final HUI chunk has been received, in indicated order:

- If TI signature verification fails: Fail with result HSM_UPDATE_FAILED
- Optionally if customer signature verification fails: Fail with result HSM_UPDATE_FAILED
- If Copy from RAM to MAIN flash fails (could indicate flash was not erased or sectors were retained during chip erase): Fail with FLASH_FSM_ERROR
- If HSM boot of new GPFW fails: Fail with result HSM_UPDATE_FAILED

11.1.10.8 Memory Clear

Relevant memory is cleared once the initial command is received and again after the command processing regardless of success or failure

11.2 SACI commands for User Record Programming

11.2.1 Program CCFG User Record - **SACI_CMD_FLASH_PROG_CCFG_USER_REC**

Program the user record part of the CCFG sector, Ccfg.userRecord.

This command can be used **after** SACI_CMD_FLASH_PROG_CCFG_SECTOR has executed with the option to skip the user record.

11.2.1.1 Restrictions

All bytes in the CCFG user record must be 0xFF before the CCFG user record programming begins.

11.2.1.2 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x0D	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved
1	31:0	key	0xB7E3A08F	Flash operation key (magic number)
33:2	31:0	data	-	Bytes to be programmed: • The first byte is in bits 7:0 of word 2 • The last byte is in bits 31:24 of word 33

11.2.1.3 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x0D	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result. One of the common results, or: - INVALID_KEY_PARAM if the key parameter is incorrect - FLASH_FSM_ERROR if the flash hardware FSM reported an error - NOT_ALLOWED if restrictions are not met (the command has no effect)
	31:24	dataWordCount	0	Size of additional response data, in number of 32-bit words

11.2.1.4 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED
- If key is invalid: Fail with result INVALID_KEY_PARAM

11.2.2 Get CCFG User Record - **SACI_CMD_MISC_GET_CCFG_USER_REC**

IMPLEMENTED

Get the readable part of the user record in CCFG, up to 128 bytes (in increments of 16 bytes).

The "non-readable part" of Ccfg.userRecord is specified by Ccfg.flashProt.readProt.ccfgSector. The read protection starts at the end of the CCFG sector, and is specified in increments of 16 bytes.

The "non-readable part" can be read by a bootloader, before CCFG restrictions are applied, but not by the application.

11.2.2.1 Availability

This command is not available in the P1STBDAY lifecycle state (result is INVALID_CMD_ID).

11.2.2.2 Restrictions

CCFG must be valid.

11.2.2.3 Parameter Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x04	Command ID
	15:8	respSeqNumber	User defined	Optional response sequence number, included in the response header
	31:16	reserved0	0x0000	Reserved

11.2.2.4 Response Words

Words	Bits	Field	Value	Description
0	7:0	cmdId	0x04	Command ID, copied from the first command parameter word
	15:8	respSeqNumber	User defined	Optional sequence number, copied from the first command parameter word
	23:16	result	-	Command result (no special result codes for this command)
	31:24	dataWordCount	0-32	Size of additional response data, in number of 32-bit words: 32 = 128 bytes if the entire user record is readable 0 = 0 bytes if the entire user record is non-readable, or restrictions are not met
1+	31:0	data	-	Readable user record data.

11.2.2.5 Check Sequence

These checks are performed before command execution, in the indicated order:

- If any restriction is violated: Fail with result NOT_ALLOWED